给我一份直接插入排序的笔记。

直接插入排序（Direct Insertion Sort）笔记

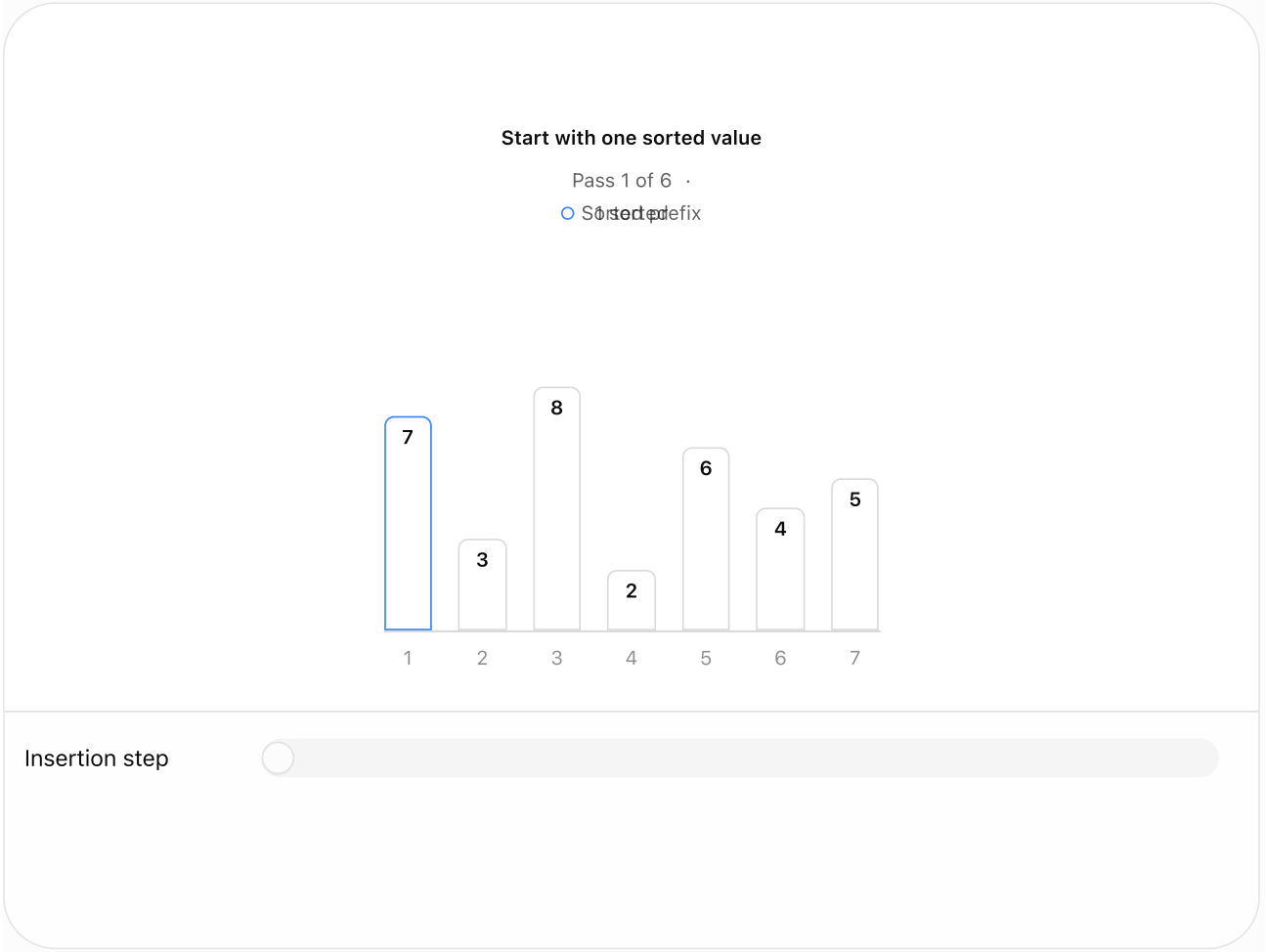
一、基本概念

直接插入排序：每次将一个待排序元素插入到前面已经有序的子序列中的适当位置，直到所有元素有序。

核心思想：**将数组分为有序区和无序区，每次从无序区取出第一个元素，插入有序区。**

- **初始状态**：第一个元素默认有序，其余元素属于无序区。
- **每趟操作**：取出无序区的第一个元素，依次与有序区中的元素比较，将比它大的元素向右移动，再将它插入空出的位置。
- **结束条件**：无序区为空，整个数组有序。

1. 排序过程示意



二、算法执行过程

以数组 `[5, 2, 4, 6, 1, 3]` 为例，对其进行升序排序。

初始状态：

```
纯文本
```

```
[5 | 2, 4, 6, 1, 3]
 ↑
有序区
```

竖线左侧为有序区，右侧为无序区。

第 1 趟：插入 2

将 2 与 5 比较，5 大于 2，因此将 5 右移，再将 2 插入空位。

```
纯文本
```

```
[2, 5 | 4, 6, 1, 3]
```

第 2 趟：插入 4

4 小于 5，将 5 右移；4 大于 2，停止移动，将 4 插入。

```
纯文本
```

```
[2, 4, 5 | 6, 1, 3]
```

第 3 趟：插入 6

6 大于 5，无须移动。

```
纯文本
```

```
[2, 4, 5, 6 | 1, 3]
```

第 4 趟：插入 1

有序区中的 6、5、4、2 依次右移，将 1 插入最前面。

```
纯文本
```

```
[1, 2, 4, 5, 6 | 3]
```

第 5 趟：插入 3

6、5、4 依次右移，3 插入 2 和 4 之间。

```
纯文本
```

```
[1, 2, 3, 4, 5, 6]
```

最终，数组完成升序排序。

关键理解

直接插入排序不是每次交换两个元素，而是：

1. 暂存待插入元素。
2. 将有序区中比它大的元素依次向右移动。
3. 将待插入元素放入空出来的位置。

注意：**右移操作必须从右向左进行比较，防止尚未处理的元素被覆盖。**

三、算法实现

1. C 语言代码

```c
void InsertSort(int A[], int n) {
    int i, j, temp;

    for (i = 1; i < n; i++) {
        temp = A[i];        // 暂存待插入元素
        j = i - 1;

        while (j >= 0 && A[j] > temp) {
            A[j + 1] = A[j]; // 将较大元素右移
            j--;
        }

        A[j + 1] = temp;    // 插入到正确位置
    }
}
```

2. 代码逐行理解

代码	含义
`for (i = 1; i < n; i++)`	从第二个元素开始，依次插入有序区
`temp = A[i]`	暂存待插入元素，防止其被覆盖
`j = i - 1`	从有序区最后一个元素开始比较
`A[j] > temp`	判断当前元素是否应该向右移动
`A[j + 1] = A[j]`	将较大的元素向右移动一位
`j--`	继续向左比较
`A[j + 1] = temp`	将待插入元素放入正确位置

为什么最后是 `A[j + 1] = temp`，而不是 `A[j] = temp`？

因为循环结束时有两种情况：

- `j >= 0`：此时 `A[j] <= temp`，所以应插入到 `j + 1` 位置。
- `j < 0`：说明有序区所有元素都比 `temp` 大，元素全部右移，应该插入下标 `0`。

因此统一使用 `A[j + 1] = temp`。

四、算法性能分析

性能指标	结论	说明
最好时间复杂度	$O(n)$	原数组已经有序，每趟只需比较一次
最坏时间复杂度	$O(n^2)$	原数组逆序，每个元素都需要大量移动
平均时间复杂度	$O(n^2)$	平均情况下需要进行平方级别的比较和移动
空间复杂度	$O(1)$	只使用常数个辅助变量
稳定性	稳定	相同元素的相对次序不变

1. 时间复杂度为什么是 $O(n^2)$?

最坏情况下，原数组为逆序：

```
纯文本
[5, 4, 3, 2, 1]
```

每次插入时，都需要将前面的元素向右移动。

总移动次数为：

$$1 + 2 + 3 + \cdots + (n-1) = \frac{n(n-1)}{2}$$

因此：

$$T(n) = O(n^2)$$

最好情况下，原数组已经升序排列，每趟只进行一次关键字比较，时间复杂度为：

$$T(n) = O(n)$$

2. 为什么直接插入排序是稳定的?

假设数组中有两个相同的元素：

```
纯文本
[3_a, 2, 3_b]
```

其中 3_a 和 3_b 的关键字相同。

代码使用的判断条件是：

```
C
A[j] > temp
```

只有严格大于待插入元素时，才会向右移动。

因此，遇到相等的元素时不会越过它，排序后仍然是：

```
纯文本
[2, 3_a, 3_b]
```

相同元素的相对次序不变，所以直接插入排序是**稳定排序**。

注意：如果改成 `A[j] >= temp`，相同元素可能改变相对次序，算法就不再保证稳定。

五、直接插入排序的核心性质

1. 循环不变式

每一趟排序开始时：

下标 `0` 到 `i-1` 的元素已经有序。

本趟将 `A[i]` 插入前面的有序区后：

- `A[0]` 到 `A[i]` 变为有序。
- 尚未处理的元素保持在无序区。
- 已有序的前缀长度增加 1。

当 `i = n` 时，整个数组有序。

2. 插入过程中会发生什么？

假设当前数组为：

```
纯文本
[2, 4, 6, 8 | 5]
```

暂存 `5` 后：

```
纯文本
[2, 4, 6, 8 | temp=5]
```

将 8、6 依次右移：

```
纯文本
[2, 4, 6, 8, 8]
[2, 4, 6, 6, 8]
```

将 `temp=5` 放入空位：

```
纯文本
[2, 4, 5, 6, 8]
```

注意：右移过程中数组可能暂时出现重复值，这是正常的，因为待插入元素已经保存在 `temp` 中。

六、直接插入排序与其他排序算法比较

排序算法	最好时间复杂度	平均时间复杂度	最坏时间复杂度	稳定性
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	稳定
冒泡排序	$O(n)$*	$O(n^2)$	$O(n^2)$	稳定
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	不稳定
希尔排序	取决于增量序列	取决于增量序列	取决于增量序列	不稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	稳定

* 冒泡排序的最好时间复杂度为 $O(n)$，前提是实现中设置了提前结束标志。

直接插入排序的适用场景

- 待排序元素数量较少。
- 原数组基本有序。
- 希望算法实现简单、额外空间占用少。
- 需要稳定排序。

七、考研 408 高频考点

考点 1：直接插入排序的趟数

对于含有 n 个元素的数组：

$$\boxed{\text{排序趟数} = n - 1}$$

因为第一个元素默认构成有序区，剩余 $n-1$ 个元素依次插入。

考点 2：每趟排序后，有序区长度是多少？

第 i 趟结束后，有序区长度为：

$$\boxed{i + 1}$$

这里规定第一趟从 $i = 1$ 开始计数。

例如，初始数组有 6 个元素：

排序阶段	有序区长度
初始状态	1
第 1 趟结束	2
第 2 趟结束	3
第 3 趟结束	4
第 4 趟结束	5
第 5 趟结束	6

考点 3：直接插入排序与逆序对

在直接插入排序中，每次将元素向左插入，本质上是在消除它与前面较大元素之间的逆序关系。

对于标准的直接插入排序实现：

$$\boxed{\text{元素移动次数} = \text{初始逆序对数量}}$$

这里的移动次数指有序区元素右移的次数，不包含将 `temp` 写回数组的操作。

因此，初始逆序对越少，直接插入排序通常越快。

考点 4：排序算法是否具有自适应性？

具有自适应性。

直接插入排序的运行时间会受到初始序列有序程度的影响：

- 已经有序：$O(n)$。
- 基本有序：通常较快。
- 完全逆序：$O(n^2)$。

考点 5：是否需要额外数组？

不需要。

直接插入排序只使用 `temp` 等少量辅助变量，属于原地排序，辅助空间复杂度为 $O(1)$。

八、考前速记版

如果考前时间紧张，重点记住以下内容：

记忆项目	结论
核心思想	将元素插入前面的有序子序列
初始有序区	第一个元素
排序趟数	$n-1$
最好时间复杂度	$O(n)$
平均时间复杂度	$O(n^2)$
最坏时间复杂度	$O(n^2)$
空间复杂度	$O(1)$
稳定性	稳定
是否原地排序	是
是否自适应	是
逆序情况下	移动次数最多

一句话记忆：

> 直接插入排序：前面有序、后面无序；逐个取出、向右移动、插入空位；最好 $O(n)$，平均和最坏 $O(n^2)$，稳定、原地、自适应。

408 做题时尤其注意： 给出某一趟排序后的数组，让你判断下一趟结果；或者让你根据排序后的有序前缀，判断当前执行到了第几趟。遇到这类题，先找出已经有序的前缀，再把无序区第一个元素插入进去即可。

给我折半插入排序的笔记。

折半插入排序（Binary Insertion Sort）笔记

一、基本概念

折半插入排序：在直接插入排序的基础上，使用折半查找（二分查找）确定待插入元素在有序区中的插入位置，再将该位置之后的元素依次向右移动，最后插入待排序元素。

核心思想：

折半查找确定位置，元素移动完成插入。

与直接插入排序一样，折半插入排序也将数组划分为有序区和无序区：

- **有序区**：已经完成排序的元素。
- **无序区**：尚未插入有序区的元素。
- **每趟操作**：取出无序区第一个元素，通过折半查找确定插入位置，再移动元素并完成插入。

1. 与直接插入排序的区别

比较项目	直接插入排序	折半插入排序
查找插入位置	从右向左逐个比较	使用折半查找
查找方式	顺序查找	折半查找
查找比较次数	最坏为 $O(n)$	最坏为 $O(\log n)$
元素移动次数	最坏为 $O(n)$	最坏为 $O(n)$
总体平均时间复杂度	$O(n^2)$	$O(n^2)$
最好时间复杂度	$O(n)$	$O(n \log n)$
空间复杂度	$O(1)$	$O(1)$
稳定性	稳定	稳定

考研重点：折半插入排序减少的是查找插入位置时的比较次数，并没有减少元素移动次数的渐进复杂度，因此总体时间复杂度仍然是 $O(n^2)$。

二、算法执行过程

以数组 `[5, 2, 4, 6, 1, 3]` 为例，进行升序排序。

第 1 趟：插入 2

初始状态：

```
纯文本
[5 | 2, 4, 6, 1, 3]
```

有序区为 `[5]`，待插入元素为 `2`。

通过折半查找，确定 `2` 应插入下标 `0`。

将 `5` 向右移动，再插入 `2`：

```
纯文本
[2, 5 | 4, 6, 1, 3]
```

第 2 趟：插入 4

当前有序区：

```
纯文本
[2, 5 | 4, 6, 1, 3]
```

待插入元素为 `4`。

折半查找过程：

1. `low = 0`，`high = 2`，`mid = 1`。
2. `A[1] = 5 > 4`，因此在左半部分继续查找。
3. 更新 `high = 1`，`mid = 0`。
4. `A[0] = 2 <= 4`，因此更新 `low = 1`。
5. 此时 `low = high = 1`，确定插入位置为下标 `1`。

将 `5` 向右移动，再插入 `4`：

```
纯文本
[2, 4, 5 | 6, 1, 3]
```

第 3 趟：插入 6

折半查找确定 `6` 应插入下标 `3`，无须移动元素。

```
纯文本
[2, 4, 5, 6 | 1, 3]
```

第 4 趟：插入 1

折半查找确定 `1` 应插入下标 `0`。

将有序区的所有元素依次向右移动：

```
纯文本
[1, 2, 4, 5, 6 | 3]
```

第 5 趟：插入 3

折半查找确定 `3` 应插入下标 `2`。

将 `4、5、6` 依次向右移动，再插入 `3`：

```
纯文本
[1, 2, 3, 4, 5, 6]
```

最终，整个数组完成升序排序。

三、算法实现

1. C 语言代码

```c
C
void BinaryInsertSort(int A[], int n) {
    int i, j, low, high, mid, temp;

    for (i = 1; i < n; i++) {
        temp = A[i];

        low = 0;
        high = i;

        // 折半查找插入位置
        while (low < high) {
            mid = low + (high - low) / 2;

            if (A[mid] <= temp)
                low = mid + 1;
            else
                high = mid;
        }

        // 将插入位置之后的元素依次右移
        for (j = i; j > low; j--) {
            A[j] = A[j - 1];
```

2. 代码逐行理解

代码	含义
`temp = A[i]`	暂存待插入元素
`low = 0`	有序区的起始下标
`high = i`	有序区的右边界（不包含）
`mid = low + (high - low) / 2`	计算中间位置
`A[mid] <= temp`	中间元素小于等于待插入元素，继续向右查找
`A[mid] > temp`	中间元素大于待插入元素，继续向左查找
`low == high`	查找结束，确定插入位置
`A[j] = A[j - 1]`	将元素向右移动一位
`A[low] = temp`	将待插入元素放入正确位置

3. 为什么 `high = i`？

当本趟待插入元素为 `A[i]` 时，有序区只包含：

$$A[0], A[1], \ldots, A[i-1]$$

插入位置可能是 `0` 到 `i` 中的任意一个下标。

因此，使用左闭右开区间：

$$\boxed{[low, high) = [0, i)}$$

这里 `high = i` 不代表 `A[i]` 属于有序区，而是表示有序区的右边界不包含 `i`。

这种写法可以统一处理所有情况，包括插入到最前面和最后面。

四、为什么使用折半查找?

这是折半插入排序最重要的知识点之一。

假设当前有序区为：

```
纯文本
[2, 4, 6, 8, 10, 12, 14, 16]
```

现在要插入 `11`。

1. 直接插入排序

从右向左逐个比较：

```
纯文本
16 → 14 → 12 → 10
```

找到 `10 < 11` 后，确定插入位置。

需要进行多次比较。

2. 折半插入排序

利用有序区的有序性，每次只需检查中间元素：

```
纯文本
[2, 4, 6, 8, 10, 12, 14, 16]
                ↑
             中间元素
```

中间元素为 `10`，因为 `10 < 11`，所以在右半部分继续查找。

随后检查右半部分的中间元素，逐步缩小查找范围，最终确定插入位置。

折半查找的比较次数为 $O(\log n)$，而顺序查找最坏需要 $O(n)$ 次比较。

但是，确定位置后，仍然需要将 `12、14、16` 向右移动。因此，折半插入排序并没有消除元素移动的开销。

五、算法性能分析

1. 时间复杂度

情况	时间复杂度	原因
最好情况	$O(n \log n)$	原序列已经有序，不需要移动元素，但仍需进行折半查找
平均情况	$O(n^2)$	查找位置较快，但元素移动仍然可能需要平方级时间
最坏情况	$O(n^2)$	逆序序列需要大量元素移动

为什么最好情况下也是 $O(n \log n)$?

这是容易与直接插入排序混淆的地方。

对于已经有序的数组：

```
纯文本
[1, 2, 3, 4, 5, 6]
```

直接插入排序每趟只需要进行一次关键字比较，因此最好时间复杂度为：

$$O(n)$$

而折半插入排序仍然会对每个待插入元素执行折半查找。

查找次数大致为：

$$\log_2 1 + \log_2 2 + \cdots + \log_2 (n-1)$$

因此，最好时间复杂度为：

$$\boxed{O(n \log n)}$$

2. 空间复杂度

折半插入排序只使用 `temp`、`low`、`high`、`mid` 等有限个辅助变量，不需要额外数组。

因此：

$$\boxed{S(n) = O(1)}$$

3. 稳定性

标准折半插入排序是稳定排序。

关键在于插入位置的确定方式。

代码中使用：

```c
C
if (A[mid] <= temp)
    low = mid + 1;
else
    high = mid;
```

当遇到与待插入元素相等的元素时，查找会继续向右，最终将待插入元素插入到已有相等元素的后面。

例如：

```
纯文本
原序列：[3ₐ, 3ᵦ, 2]
```

排序后：

```
纯文本
[2, 3ₐ, 3ᵦ]
```

相同元素的相对次序没有改变，因此算法稳定。

注意：如果将相等时的处理方式改为优先插入到已有相等元素之前，就可能破坏稳定性。

六、折半插入排序与直接插入排序的核心区别

这部分建议重点记忆。

比较项目	直接插入排序	折半插入排序
查找方式	从右向左逐个比较	折半查找
查找插入位置	最坏 $O(n)$	最坏 $O(\log n)$
元素移动	最坏 $O(n)$/趟	最坏 $O(n)$/趟
最好时间复杂度	$O(n)$	$O(n \log n)$
平均时间复杂度	$O(n^2)$	$O(n^2)$
最坏时间复杂度	$O(n^2)$	$O(n^2)$
空间复杂度	$O(1)$	$O(1)$
稳定性	稳定	稳定
是否原地排序	是	是

一句话理解

- **直接插入排序**：边比较，边寻找插入位置，边移动元素。
- **折半插入排序**：先通过折半查找确定插入位置，再统一移动元素。

七、考研 408 高频考点

考点 1：折半插入排序能否减少移动次数？

不能从渐进复杂度上减少移动次数。

折半查找只能减少比较次数，不能改变待插入元素之前有多少个元素需要向右移动。

例如，将 `1` 插入有序序列：

纯文本

```
[2, 4, 6, 8, 10]
```

折半查找可以快速确定插入位置为 `0`，但是仍然需要移动全部 5 个元素。

考点 2：为什么平均时间复杂度仍然是 $O(n^2)$?

每一趟插入最多需要移动 $O(n)$ 个元素，总共需要进行 $n-1$ 趟插入。

因此，元素移动的总次数在平均情况下仍然是平方级的：

$$\boxed{T(n) = O(n^2)}$$

考点 3：折半插入排序是稳定排序吗?

是，标准实现稳定。

判断关键不是使用了折半查找还是顺序查找，而是看相等元素的相对次序是否会发生改变。

考点 4：折半插入排序是原地排序吗?

是。

因为它直接修改原数组，只使用常数个辅助变量，所以空间复杂度为 $O(1)$。

考点 5：折半插入排序适合什么场景?

适用于需要减少关键字比较次数的场景，尤其是关键字比较成本较高时。

但由于元素移动仍然可能很多，所以它并不一定比直接插入排序更快。

八、考前速记版

知识点	必须记住的结论
核心思想	折半查找插入位置，再移动元素
查找复杂度	每趟最坏 $O(\log n)$
最好时间复杂度	$O(n \log n)$
平均时间复杂度	$O(n^2)$
最坏时间复杂度	$O(n^2)$
空间复杂度	$O(1)$
稳定性	稳定
原地排序	是
相比直接插入排序的优势	减少关键字比较次数
相比直接插入排序的局限	不能减少元素移动次数的渐进复杂度

最后记住这句话：

> 折半插入排序用折半查找确定插入位置，但仍需要移动元素；最好时间复杂度为 $O(n \log n)$，平均和最坏时间复杂度均为 $O(n^2)$，空间复杂度为 $O(1)$，且稳定、原地。

有没有和上面插入排序类似的动画可以给我？就是可以互动的那个东西。

有！而且我找到了几个很适合你复习考研 408 的交互式排序动画。🎯

1. 最推荐：可以逐步回放的插入排序动画

Insertion Sort Visualizer

这个工具特别适合你现在的学习方式：

- **自定义数据**：输入自己的数组，例如 `5, 2, 4, 6, 1, 3`。
- **逐步执行**：每次只看一步，观察元素如何移动。
- **前进与后退**：可以反复检查某一趟排序。
- **播放与暂停**：自己控制学习速度。
- **查看排序步骤**：了解每个元素是怎样插入有序区的。

建议你先输入 `5, 2, 4, 6, 1, 3`，逐步观察每一趟的变化。

2. 适合学习多种排序算法

EasySorting 交互式排序可视化

它支持多种经典排序算法，包括：

- 直接插入排序
- 冒泡排序
- 简单选择排序
- 希尔排序
- 快速排序
- 归并排序
- 堆排序
- 基数排序

你可以输入自己的数组、单步执行，也可以查看不同语言的代码实现。

它适合你后续整理整个 408 数据结构排序章节时使用。

3. 可以观察比较和移动过程

Sorting Race 排序动画

这个工具提供直接插入排序和冒泡排序的动态演示，还可以：

- 调整数组规模。
- 调节动画速度。
- 暂停或重新打乱数组。
- 观察比较次数和交换次数的变化。

它比较适合直观理解不同排序算法的执行过程。

---

4. 关于你刚学的折半插入排序

这里需要特别说明一下：**上面这些工具主要展示直接插入排序，并不一定支持折半插入排序。**

两者最重要的区别是：

- 直接插入排序：逐个比较，寻找插入位置。
- 折半插入排序：先用折半查找确定位置，再移动元素。

如果你希望像刚才的动画那样，能够单步控制、观察 `low`、`high`、`mid` 的变化，同时看见元素右移，那么专门制作一个折半插入排序交互动画会更贴合 408 的考点。

我也可以直接在这里为你生成一个**可交互的折半插入排序演示**，让你逐步查看每次二分查找、插入位置和元素移动的全过程。

直接插入排序就是将无序区取到的元素，逐个与有序区的元素进行比较；而折半插入排序就是从无序区取一个元素，然后利用折半查找算法查找其在有序区的大致位置，再进行比较，对不对？

你的理解**基本正确！**👍 只有一个细节需要纠正：折半插入排序不是先找一个“大致位置”，而是通过折半查找，**直接确定准确的插入位置**。

一、两者的核心区别

1. 直接插入排序

从无序区取出一个元素，与有序区的元素逐个比较，边比较边移动，直到找到合适的插入位置。

例如，有序区为：

```
纯文本
[2, 4, 6, 8] | 5
```

插入 `5`：

```
纯文本
5 与 8 比较 → 8 右移
5 与 6 比较 → 6 右移
5 与 4 比较 → 找到插入位置
```

结果：

```
纯文本
[2, 4, 5, 6, 8]
```

2. 折半插入排序

同样从无序区取出一个元素，但先通过折半查找确定插入位置，然后统一移动元素。

还是这个例子：

```
纯文本
[2, 4, 6, 8] | 5
```

折半查找过程：

1. 比较中间元素 `4` 与 `5`，发现 `4 < 5`，在右半部分查找。
2. 比较 `6` 与 `5`，发现 `6 > 5`，在左半部分查找。
3. 确定 `5` 应插入 `4` 和 `6` 之间。

然后将 `6、8` 依次右移：

```
纯文本
[2, 4, 5, 6, 8]
```

二、用一句话区分

对比	直接插入排序	折半插入排序
如何找位置	逐个比较	折半查找
什么时候移动元素	查找过程中逐个移动	找到位置后统一移动
比较次数	较多	较少
移动次数	可能较多	可能较多
平均时间复杂度	$O(n^2)$	$O(n^2)$

考研 408 记忆口诀：

直接插入：边比较、边移动、找位置。
折半插入：先二分、定位置、再移动。

还有一个关键点：**折半插入排序的折半查找不是在整个数组中进行，而是在当前已经有序的部分中进行。**

给我希尔排序的笔记。

希尔排序（Shell Sort）笔记

一、基本概念

希尔排序又称**缩小增量排序**，是对直接插入排序的一种改进。

它的核心思想是：

先将间隔较远的元素分组，对各组分别进行直接插入排序；再逐渐缩小增量，使元素逐步接近最终位置；最后以增量 1 进行直接插入排序。

希尔排序的关键不是换了一种插入方法，而是通过分组插入排序，让元素能够一次移动较远的距离，从而减少后续的移动次数。

1. 为什么需要希尔排序?

假设数组为：

```
纯文本
[8, 7, 6, 5, 4, 3, 2, 1]
```

如果直接使用插入排序，由于数组是逆序的，元素需要频繁移动。

希尔排序则先按一定间隔分组，让距离较远的元素先进行调整，逐步改善整个数组的有序程度。

2. 核心概念

概念	含义
增量（gap）	同一组中相邻元素的下标间隔
分组	根据下标对增量取模，将元素划分到不同组
分组插入排序	每组内部进行直接插入排序
缩小增量	逐渐减小 gap，让元素更接近最终位置
最后一趟	gap 必须为 1，相当于对整个数组进行直接插入排序

注意：希尔排序不是每趟都对整个数组进行普通的直接插入排序，而是按照当前增量，对多个子序列分别进行直接插入排序。

二、希尔排序的执行过程

假设有数组：

```
纯文本
```

```
[8, 7, 6, 5, 4, 3, 2, 1]
```

为了方便理解，采用常见的增量序列：

$$\boxed{gap = 4,\ 2,\ 1}$$

注意：这只是其中一种增量序列，并不是所有希尔排序都必须采用这种方式。

第一趟：gap = 4

下标相差 4 的元素属于同一组。

分组如下：

```
纯文本
```

```
第 1 组：A[0], A[4] → [8, 4]
第 2 组：A[1], A[5] → [7, 3]
第 3 组：A[2], A[6] → [6, 2]
第 4 组：A[3], A[7] → [5, 1]
```

分别对每组进行直接插入排序：

```
纯文本
```

```
第 1 组：[8, 4] → [4, 8]
第 2 组：[7, 3] → [3, 7]
第 3 组：[6, 2] → [2, 6]
第 4 组：[5, 1] → [1, 5]
```

重新放回原数组：

```
纯文本
```

```
初始：[8, 7, 6, 5, 4, 3, 2, 1]
结果：[4, 3, 2, 1, 8, 7, 6, 5]
```

可以看到，较小的元素已经向数组前面移动了一大段距离。

第二趟：gap = 2

现在下标相差 2 的元素属于同一组。

```
纯文本
```

对应的元素为：

```
纯文本

第 1 组：[4, 2, 8, 6]
第 2 组：[3, 1, 7, 5]
```

分别进行直接插入排序：

```
纯文本

第 1 组：[4, 2, 8, 6] → [2, 4, 6, 8]
第 2 组：[3, 1, 7, 5] → [1, 3, 5, 7]
```

重新放回原数组：

```
纯文本

[2, 1, 4, 3, 6, 5, 8, 7]
```

此时数组还没有整体有序，但元素已经进一步接近自己的最终位置。

第三趟：gap = 1

当增量为 1 时，相邻元素属于同一组。

此时只有一个组：

```
纯文本

[2, 1, 4, 3, 6, 5, 8, 7]
```

对整个数组进行一次普通的直接插入排序：

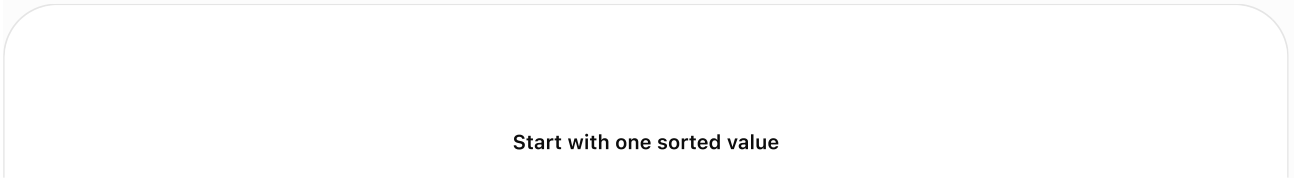
```
纯文本

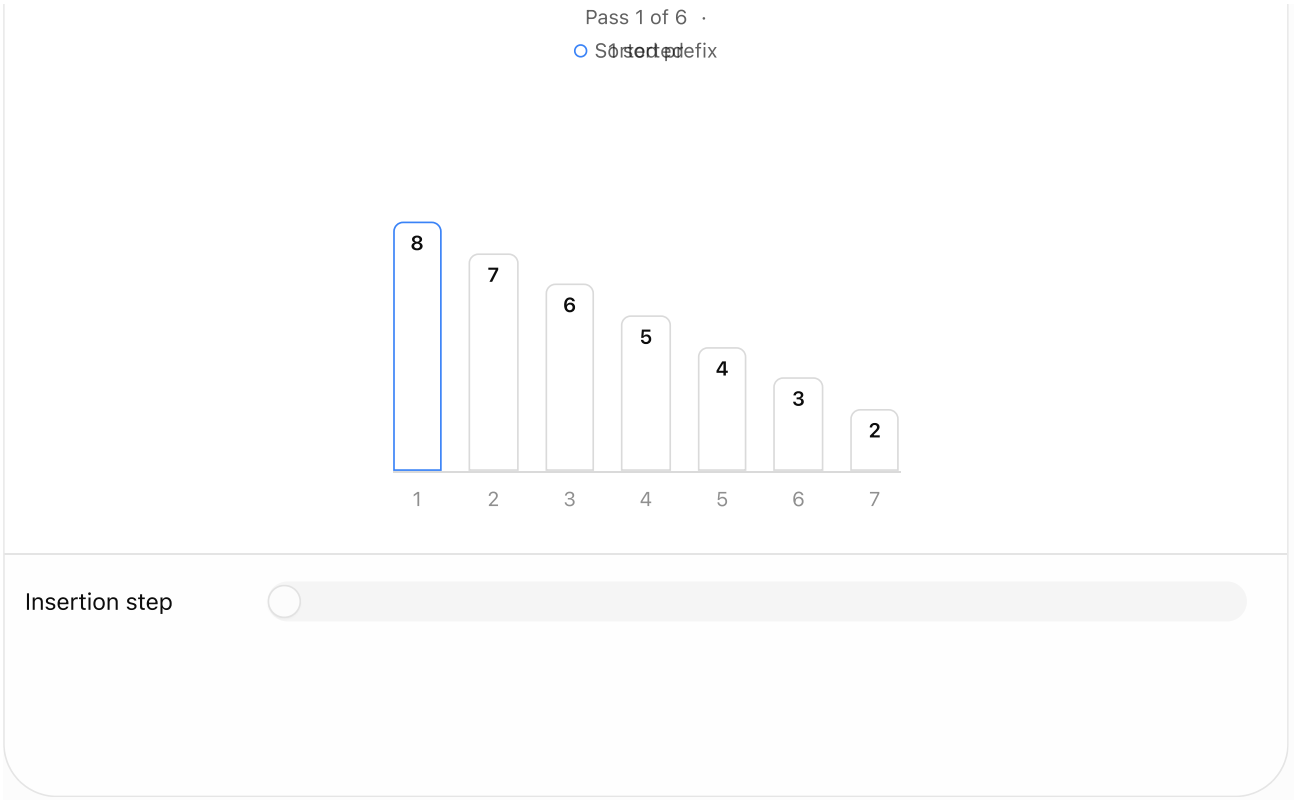
[1, 2, 3, 4, 5, 6, 7, 8]
```

排序完成！

交互式理解

下面这个动画可以帮助你理解插入排序的基本操作。观察希尔排序时，可以重点关注：**每趟的 gap 如何决定哪些元素参与同一组的插入排序。**





这个动画展示的是直接插入排序，并非完整的希尔排序动画；上面的 `gap = 4、2、1` 才是本例希尔排序的完整过程。

三、算法实现

1. C 语言代码

C

```
void ShellSort(int A[], int n) {
```

这里使用的是**希尔提出的简单增量序列**：

$$\left\lfloor \frac{n}{2} \right\rfloor, \left\lfloor \frac{n}{4} \right\rfloor, \ldots, 2, 1$$

代码中每次将 `gap` 整除 2，直到增量变为 0 后退出循环。

2. 关键代码理解

代码	含义
`gap = n / 2`	初始化增量
`gap /= 2`	逐渐缩小增量
`i = gap`	从第二个分组位置开始处理
`temp = A[i]`	暂存待插入元素
`j = i - gap`	从同组前一个元素开始比较
`A[j + gap] = A[j]`	将同组中较大的元素向后移动一个增量
`j -= gap`	沿当前分组向前比较
`A[j + gap] = temp`	将待插入元素放回正确位置

3. 为什么不是 `j--`，而是 `j -= gap`？

这是理解希尔排序代码的关键。

普通直接插入排序中，相邻元素的下标差为 1，因此每次向前移动一个位置：

```
C
j--;
```

希尔排序中，同组元素的下标差为 `gap`，所以每次必须向前移动一个增量：

```
C
j -= gap;
```

同理，移动元素时也不是：

```
C
```

而是：

```c
A[j + gap] = A[j];
```

记忆方法：希尔排序只是把直接插入排序中的步长 1 改成了 gap。

四、算法性能分析

希尔排序的性能与**增量序列的选择**密切相关，因此不能简单地给所有希尔排序实现指定同一个时间复杂度。

性能指标	结论
最好时间复杂度	取决于增量序列及具体实现
平均时间复杂度	取决于增量序列，通常不能直接给出统一结论
最坏时间复杂度	取决于增量序列
空间复杂度	$O(1)$
稳定性	不稳定
是否原地排序	是

1. 为什么希尔排序是不稳定的?

希尔排序允许相距较远的元素直接交换相对位置，因此相同关键字的元素可能改变原有次序。

例如，两个关键字相同的元素：

```纯文本
[2ₐ, 2ᵦ, 1]
```

如果先以 `gap = 2` 进行分组插入排序，`2ₐ` 与 `1` 属于同一组。将 `1` 插入前面时，数组可能变为：

```纯文本
[1, 2ᵦ, 2ₐ]
```

两个相同元素的相对次序发生了改变。

因此：

$$\boxed{\text{希尔排序是不稳定排序}}$$

2. 为什么希尔排序通常比直接插入排序快?

主要有两个原因:

1. **先处理远距离元素**:让元素可以一次向前移动多个位置。
2. **逐步改善有序程度**:最后 `gap = 1` 时,整个数组通常已经比较接近有序,最终插入排序需要的移动较少。

但要注意,希尔排序并不保证对任意输入、任意增量序列都比直接插入排序快。

五、希尔排序与直接插入排序的比较

比较项目	直接插入排序	希尔排序
核心思想	逐个插入有序区	分组插入,逐步缩小增量
分组	不分组	按增量分组
元素移动距离	每次移动 1 个位置	每次可以移动多个位置
最后一趟	普通插入排序	`gap = 1`,普通插入排序
时间复杂度	平均 $O(n^2)$	取决于增量序列
空间复杂度	$O(1)$	$O(1)$
稳定性	稳定	不稳定
原地排序	是	是

六、考研 408 高频考点

考点 1:希尔排序的最后一个增量必须是多少?

$$\boxed{gap = 1}$$

只有当增量为 1 时,才能保证最终对整个数组进行一次直接插入排序,使整个数组有序。

考点 2:希尔排序每趟结束后,整个数组一定有序吗?

不一定。

每趟结束后,只能保证各个分组内部有序,不能保证整个数组有序。

例如:

```
纯文本
[4, 3, 2, 1, 8, 7, 6, 5]
```

虽然这是 `gap = 4` 的一趟排序结果，各组内部已经有序，但整个数组显然还没有有序。

考点 3：希尔排序是否稳定?

$$\boxed{\text{不稳定}}$$

原因：不同分组中的元素可能跨越较远距离移动，改变相同关键字元素的相对次序。

考点 4：希尔排序的时间复杂度是多少?

必须结合增量序列判断。

例如，对于采用简单增量序列 `n/2、n/4、...、1` 的经典实现，最坏时间复杂度为 $O(n^2)$。

不能因为使用了分组，就直接认为希尔排序的时间复杂度一定是 $O(n \log n)$。

考点 5：希尔排序属于原地排序吗?

属于。

它只需要少量辅助变量，不需要额外的数组，因此空间复杂度为：

$$\boxed{O(1)}$$

七、考前速记版

知识点	结论
别名	缩小增量排序
本质	分组直接插入排序
增量变化	逐渐缩小，最终为 1
最后一趟	对整个数组进行直接插入排序
每趟是否整体有序	不一定
空间复杂度	$O(1)$
稳定性	不稳定
是否原地排序	是
时间复杂度	取决于增量序列

一句话记忆：

> 希尔排序就是按照增量分组进行直接插入排序，逐步缩小增量，最后以 `gap = 1` 对整个数组进行直接插入排序。它是原地、不稳定的排序算法，时间复杂度取决于增量序列。

408 最容易混淆的三个结论

1. **折半插入排序**：减少比较次数，但移动次数仍可能达到平方级。
2. **希尔排序**：通过分组和远距离移动，减少后续插入排序的工作量。
3. **稳定性**：直接插入排序稳定，折半插入排序可以稳定实现，希尔排序不稳定。

这三个知识点经常放在一起比较，建议直接记住。

给我冒泡排序的笔记。

冒泡排序（Bubble Sort）笔记

一、基本概念

冒泡排序：通过不断比较相邻两个元素的关键字，并在逆序时交换它们，使较小或较大的元素逐渐移动到数组的一端，最终使整个数组有序。

升序排序时，常见的实现方式是：**每趟从前向后比较相邻元素，将较大的元素逐步向后移动，使当前未排序部分的最大元素最终到达末尾。**

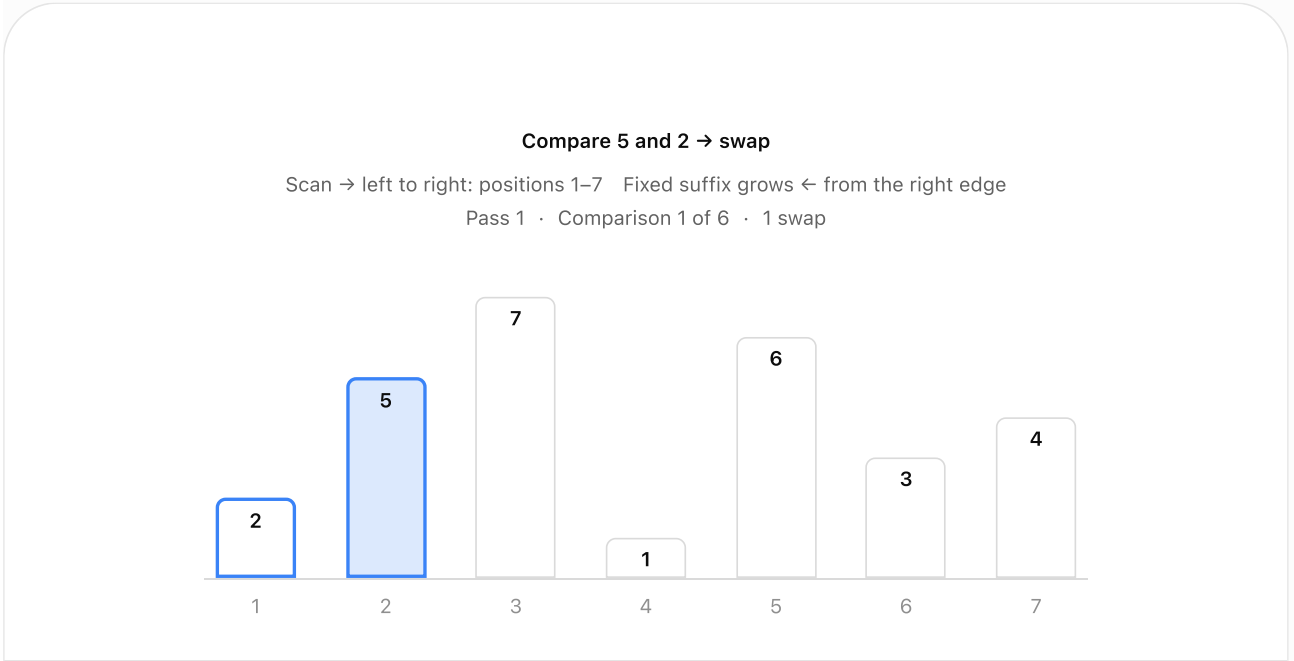
1. 核心思想

冒泡排序的主要特点：

- **比较对象**：相邻的两个元素。
- **操作方式**：如果顺序不符合要求，就交换两个元素。
- **每趟结果**：当前未排序部分的最大元素到达末尾。
- **结束条件**：所有元素有序，或者某一趟没有发生任何交换。

2. 交互式动画

可以逐步执行下面的动画，观察相邻元素的比较、交换，以及每趟结束后最大元素如何固定在右侧。



Swap

Result: 2, 5; 5 moves one place right

Sorting step

观察重点： 冒泡排序每次只比较相邻元素；每完成一趟，未排序部分的最大元素就会移动到末尾。

二、算法执行过程

以数组 `[5, 2, 4, 1, 3]` 为例，进行升序排序。

第一趟

从左向右比较相邻元素。

初始数组：

```
纯文本
[5, 2, 4, 1, 3]
```

第 1 次比较：5 和 2

因为 $5 > 2$，交换：

```
纯文本
[2, 5, 4, 1, 3]
```

第 2 次比较：5 和 4

因为 $5 > 4$，交换：

```
纯文本
[2, 4, 5, 1, 3]
```

第 3 次比较：5 和 1

因为 $5 > 1$，交换：

```
纯文本
[2, 4, 1, 5, 3]
```

第 4 次比较：5 和 3

因为 $5 > 3$，交换：

```
纯文本
```

```
[2, 4, 1, 3, 5]
```

第一趟结束，最大元素 `5` 已经到达末尾。

```
纯文本
[2, 4, 1, 3 | 5]
```

右侧的 `5` 已经确定最终位置，不需要再参与后续比较。

第二趟

继续对未排序部分 `[2, 4, 1, 3]` 从左向右比较。

```
纯文本
[2, 4, 1, 3, 5]
```

比较 `2` 和 `4`：不交换。

比较 `4` 和 `1`：交换。

```
纯文本
[2, 1, 4, 3, 5]
```

比较 `4` 和 `3`：交换。

```
纯文本
[2, 1, 3, 4, 5]
```

第二趟结束，`4` 也到达了自己的最终位置。

```
纯文本
[2, 1, 3 | 4, 5]
```

第三趟

对 `[2, 1, 3]` 进行冒泡排序。

比较 `2` 和 `1`，交换：

```
纯文本
[1, 2, 3, 4, 5]
```

比较 `2` 和 `3`，不交换。

第三趟结束。

第四趟

对剩余的 `[1, 2]` 进行比较。

因为 `1 < 2`，不发生交换。

如果使用带有提前结束标志的实现，此时就可以结束排序。

最终结果：

```
纯文本
```

```
[1, 2, 3, 4, 5]
```

三、算法实现

1. C 语言代码

```
C
```

```c
void BubbleSort(int A[], int n) {
    int i, j, temp;
    int flag;

    for (i = 0; i < n - 1; i++) {
        flag = 0;  // 标记本趟是否发生交换

        for (j = 0; j < n - 1 - i; j++) {
            if (A[j] > A[j + 1]) {
                temp = A[j];
                A[j] = A[j + 1];
                A[j + 1] = temp;

                flag = 1;
            }
        }

        // 本趟没有发生交换，说明数组已经有序
        if (flag == 0)
            return;
    }
}
```

2. 关键代码理解

代码	含义
`for (i = 0; i < n - 1; i++)`	最多进行 $n-1$ 趟排序
`j < n - 1 - i`	逐渐缩小比较范围，跳过已经确定位置的末尾元素
`A[j] > A[j + 1]`	判断相邻元素是否逆序
交换代码	将较大的元素向右移动
`flag = 1`	记录本趟发生过交换
`if (flag == 0)`	本趟没有交换，提前结束排序

3. 为什么内层循环是 `j < n - 1 - i`？

因为每完成一趟，当前未排序部分的最大元素就会到达末尾。

例如，数组有 5 个元素：

趟数	本趟需要比较的相邻元素对数	已确定位置
第 1 趟	4	最后 1 个元素
第 2 趟	3	最后 2 个元素
第 3 趟	2	最后 3 个元素
第 4 趟	1	最后 4 个元素

所以第 `i` 趟只需要比较 `n - 1 - i` 对相邻元素。

4. 为什么需要 `flag`？

`flag` 用于判断本趟排序是否发生交换。

- `flag = 1`：发生过交换，数组可能仍然无序，需要继续。
- `flag = 0`：没有发生交换，说明当前数组已经有序，可以直接结束。

例如：

```
纯文本
[1, 2, 3, 4, 5]
```

第一趟比较时，没有任何相邻元素需要交换，因此算法可以提前结束。

注意：只有带有这种提前结束判断的实现，才能保证最好时间复杂度达到 $O(n)$。

四、算法性能分析

性能指标	结论	说明
最好时间复杂度	$O(n)$	优化实现中，数组已经有序，只需进行一趟比较
平均时间复杂度	$O(n^2)$	平均需要多趟比较和交换
最坏时间复杂度	$O(n^2)$	逆序数组需要大量比较和交换
空间复杂度	$O(1)$	只使用常数个辅助变量
稳定性	稳定	相等元素不会相互交换
是否原地排序	是	直接修改原数组

1. 为什么最坏时间复杂度是 $O(n^2)$?

每趟比较次数依次为：

$$(n-1) + (n-2) + \cdots + 1$$

根据等差数列求和公式：

$$\begin{aligned} T(n) &= \frac{n(n-1)}{2} \\ &= O(n^2) \end{aligned}$$

因此，冒泡排序的平均和最坏时间复杂度均为：

$$\boxed{O(n^2)}$$

2. 为什么冒泡排序是稳定的?

关键在于代码的判断条件：

```c
if (A[j] > A[j + 1])
```

只有左边元素严格大于右边元素时才交换。

如果两个元素的关键字相等，就不会交换它们，因此相同关键字元素的相对次序保持不变。

所以：

$$\boxed{\text{冒泡排序是稳定排序}}$$

注意：如果使用 `>=` 作为交换条件，相同关键字元素就可能交换位置，算法将不再保证稳定。

五、冒泡排序的核心性质

1. 每趟结束后，最大元素到达末尾

这是升序冒泡排序最重要的性质。

例如：

纯文本

```
初始：[7, 3, 5, 2, 4]
第 1 趟：[3, 5, 2, 4, 7]
第 2 趟：[3, 2, 4, 5, 7]
第 3 趟：[2, 3, 4, 5, 7]
```

每一趟结束后，未排序部分的最大元素都会被确定到末尾。

2. 冒泡排序与直接插入排序的区别

对比项目	冒泡排序	直接插入排序
核心操作	比较相邻元素，必要时交换	将元素插入有序区
每趟确定的位置	未排序部分的最大元素到达末尾	有序区扩大一个元素
移动方式	相邻交换	元素右移，再插入
最好时间复杂度	$O(n)$，需提前结束优化	$O(n)$
平均时间复杂度	$O(n^2)$	$O(n^2)$
最坏时间复杂度	$O(n^2)$	$O(n^2)$
稳定性	稳定	稳定
空间复杂度	$O(1)$	$O(1)$

记忆方法：

- 冒泡排序：最大元素不断向右移动。
- 直接插入排序：有序区不断向右扩张。

六、考研 408 高频考点

考点 1：冒泡排序最多需要多少趟?

对于含有 n 个元素的数组，最多需要：

$$\boxed{n-1 \text{ 趟}}$$

考点 2：冒泡排序每趟需要比较多少次？

对于标准的从前向后冒泡实现：

$$\boxed{n-1-i}$$

这里 `i` 从 `0` 开始，表示已经完成的趟数。

所有趟的比较次数之和为：

$$\boxed{\frac{n(n-1)}{2}}$$

这是不考虑提前结束、完整执行所有趟时的比较次数。

考点 3：冒泡排序是否具有自适应性？

具有自适应性，但需要采用能够提前结束的实现。

当数组已经有序时，一趟比较没有发生交换，算法便可以提前结束。

因此，最好时间复杂度为 $O(n)$。

考点 4：冒泡排序与逆序对有什么关系？

对于标准冒泡排序，每次交换相邻的逆序元素，都会使逆序对数量减少 1。

因此：

$$\boxed{\text{交换次数} = \text{初始逆序对数量}}$$

这个性质适用于通过交换相邻逆序元素完成排序的标准冒泡排序。

例如：

```
纯文本
```

```
[3, 2, 1]
```

初始有 3 个逆序对：

$$(3,2),\quad (3,1),\quad (2,1)$$

因此排序过程中总共需要进行 3 次交换。

考点 5：冒泡排序是原地排序吗？

是。

只使用一个临时变量进行交换，额外空间复杂度为：

$$\boxed{O(1)}$$

七、考前速记版

知识点	结论
核心思想	比较相邻元素，逆序则交换
每趟确定的位置	当前未排序部分的最大元素到达末尾
最多排序趟数	$n-1$
最好时间复杂度	$O(n)$，需要提前结束优化
平均时间复杂度	$O(n^2)$
最坏时间复杂度	$O(n^2)$
空间复杂度	$O(1)$
稳定性	稳定
是否原地排序	是
交换次数	等于初始逆序对数量

最后记住这句话

> 冒泡排序通过比较相邻元素并交换逆序对，使未排序部分的最大元素逐趟移动到末尾。最多进行 $n-1$ 趟，平均和最坏时间复杂度为 $O(n^2)$，优化实现的最好时间复杂度为 $O(n)$，空间复杂度为 $O(1)$，且稳定、原地。

408 特别注意： 冒泡排序每趟确定的是末尾元素的位置，而直接插入排序每趟扩大的是前面的有序区。两者虽然时间复杂度相同，但排序过程和确定最终位置的方向不同。

给我快速排序的笔记。

快速排序是 408 数据结构中非常重要的排序算法，重点掌握 **一次划分的过程、递归实现、时间复杂度、空间复杂度、不稳定的原因，以及最好/最坏情况的区别**。下面按照考研复习笔记的形式整理，方便你直接放进 Obsidian。

快速排序（Quick Sort）笔记

一、基本思想

快速排序是一种基于分治法的排序算法。

核心思想：每次选择一个枢轴（基准）元素 `pivot`，通过一次划分，将待排序区间分成两部分，使得：

- 左侧元素不大于枢轴。
- 右侧元素不小于枢轴。
- 枢轴最终处于其排序后的正确位置。

然后分别对枢轴左侧和右侧的子序列进行快速排序，直到子序列长度不超过 1。

算法流程

1. 选择枢轴 `pivot`。
2. 对当前区间进行划分（Partition）。
3. 确定枢轴的最终位置。
4. 递归排序枢轴左侧的子序列。
5. 递归排序枢轴右侧的子序列。

注意：**快速排序的关键在于划分，而不是直接将整个序列排序。**

二、划分过程示例

假设待排序序列为：

$$[49,\ 38,\ 65,\ 97,\ 76,\ 13,\ 27]$$

选择第一个元素 `49` 作为枢轴。

经过一次划分，可以得到：

$$[27,\ 38,\ 13,\ \boxed{49},\ 76,\ 97,\ 65]$$

此时：

- `49` 左侧的元素均不大于 `49`。
- `49` 右侧的元素均不小于 `49`。
- `49` 已经处于最终排序位置。

但是，左右两侧内部仍然不一定有序！

接下来分别对左侧：

$$[27,\ 38,\ 13]$$

和右侧：

$$[76,\ 97,\ 65]$$

进行递归排序。

最终得到：

$$[13,\ 27,\ 38,\ 49,\ 65,\ 76,\ 97]$$

理解重点

一次划分完成后，只有枢轴元素的位置可以确定；不能认为整个序列已经有序，也不能认为枢轴左侧或右侧内部已经有序。

三、一次划分的具体过程

快速排序的划分方法有多种。408 常见教材通常采用**以第一个元素为枢轴、左右指针交替扫描的挖坑法**。

1. 基本思路：挖坑法

以序列为例：

$$[49,\ 38,\ 65,\ 97,\ 76,\ 13,\ 27]$$

初始设置：

- `pivot = 49`：保存第一个元素。
- `low = 0`：指向区间最左端。
- `high = 6`：指向区间最右端。

此时，先将 `49` 暂存起来，相当于第一个位置出现了一个空位。

第一步：从右向左寻找小于枢轴的元素。

从 `27` 开始扫描，发现：

$$27 < 49$$

将 `27` 放入左侧的空位：

$$[27,\ 38,\ 65,\ 97,\ 76,\ 13,\ \square]$$

此时，右侧出现新的空位。

第二步：从左向右寻找大于枢轴的元素。

从左向右扫描，发现 `65`：

$$65 > 49$$

将 `65` 放入右侧的空位：

$$[27,\ 38,\ \square,\ 97,\ 76,\ 13,\ 65]$$

此时，左侧出现新的空位。

第三步：再次从右向左扫描。

继续向左寻找小于枢轴的元素，发现 `13`：

$$13 < 49$$

将 `13` 填入左侧空位：

$$[27,\ 38,\ 13,\ 97,\ 76,\ \square,\ 65]$$

第四步：再次从左向右扫描。

发现 `97`：

$$97 > 49$$

将 `97` 填入右侧空位：

$$[27,\ 38,\ 13,\ \square,\ 76,\ 97,\ 65]$$

此时，`low` 与 `high` 相遇，停止扫描。

第五步：将枢轴放入最终位置。

把暂存的 `49` 放入相遇位置：

$$\boxed{[27,\ 38,\ 13,\ 49,\ 76,\ 97,\ 65]}$$

一次划分完成。

2. 挖坑法的记忆口诀

> 保存枢轴先挖坑，右边先找小于它；
> 找到填左边，再从左边找大于它；
> 找到填右边，左右交替继续找；
> 指针相遇填枢轴，划分结束位置定。

注意：以上示例中，左右指针的移动方向交替进行。使用挖坑法时，必须正确处理相遇条件以及重复元素，避免死循环或越界。

四、快速排序的 C 语言实现

1. 一次划分函数

C

```c
int Partition(int A[], int low, int high)
{
    int pivot = A[low];

    while (low < high)
    {
        // 从右向左，寻找小于枢轴的元素
        while (low < high && A[high] >= pivot)
            high--;

        A[low] = A[high];

        // 从左向右，寻找大于枢轴的元素
        while (low < high && A[low] <= pivot)
            low++;

        A[high] = A[low];
    }

    // 将枢轴放入最终位置
    A[low] = pivot;
```

2. 快速排序递归函数

C

```c
void QuickSort(int A[], int low, int high)
{
    if (low < high)
    {
        int pivotpos = Partition(A, low, high);

        QuickSort(A, low, pivotpos - 1);
        QuickSort(A, pivotpos + 1, high);
    }
}
```

调用方式：

C

```c
QuickSort(A, 0, n - 1);
```

3. 代码重点理解

代码	作用
`int pivot = A[low];`	保存枢轴元素
`while (low < high)`	左右指针尚未相遇时继续划分
`A[high] >= pivot`	从右向左跳过大于等于枢轴的元素
`A[low] <= pivot`	从左向右跳过小于等于枢轴的元素
`A[low] = pivot;`	将枢轴放入最终位置
`return low;`	返回枢轴的最终下标
`QuickSort(A, low, pivotpos - 1)`	递归排序左子序列
`QuickSort(A, pivotpos + 1, high)`	递归排序右子序列

考研易错点： 枢轴已经处于最终位置，因此递归时不能再把枢轴包含进去。

正确写法：

```c
QuickSort(A, low, pivotpos - 1);
QuickSort(A, pivotpos + 1, high);
```

错误写法：

```c
QuickSort(A, low, pivotpos);
QuickSort(A, pivotpos, high);
```

后一种写法可能导致区间无法缩小，从而引起无限递归。

五、时间复杂度分析

快速排序的时间复杂度取决于每次划分后，左右子序列的长度是否均衡。

1. 最好情况：$O(n \log n)$

每次划分都比较均衡，枢轴将序列分成长度大致相等的两部分。

例如：

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

每一层递归处理的元素总数约为 n，而递归层数约为：

$$\log_2 n$$

因此：

$$\boxed{T(n) = O(n \log n)}$$

2. 最坏情况：$O(n^2)$

每次划分都极不均衡，例如一侧有 $n-1$ 个元素，另一侧没有元素。

此时：

$$T(n) = T(n-1) + O(n)$$

展开：

$$T(n) = n + (n-1) + (n-2) + \cdots + 1$$

因此：

$$T(n) = \frac{n(n+1)}{2}$$

所以：

$$\boxed{T(n) = O(n^2)}$$

当序列已经有序，而每次都选择第一个元素作为枢轴时，就可能出现这种最坏情况。

3. 平均情况：$O(n \log n)$

在通常的随机输入或随机选择枢轴的分析模型下，快速排序的平均时间复杂度为：

$$\boxed{O(n \log n)}$$

注意：**平均时间复杂度为 $O(n \log n)$，不代表任何输入都能达到这个复杂度。**

六、空间复杂度

快速排序通常采用原地划分，不需要额外开辟与原数组等大的辅助数组。

但是，它需要递归调用栈，因此空间复杂度并非严格意义上的 $O(1)$。

情况	递归深度	辅助空间复杂度
最好情况	$O(\log n)$	$O(\log n)$
平均情况	$O(\log n)$	$O(\log n)$
最坏情况	$O(n)$	$O(n)$

因此，通常记为：

- 平均空间复杂度：$O(\log n)$。
- 最坏空间复杂度：$O(n)$。

这里的空间复杂度主要指递归调用栈所占用的辅助空间，不包括原数组。

七、稳定性与其他性质

1. 稳定性：不稳定

快速排序是不稳定的排序算法。

稳定性要求：如果两个元素的关键字相同，那么排序前后它们的相对次序不发生改变。

例如，两个元素的关键字相同，但分别带有原始标记：

$$[2_a,\ 2_b,\ 1]$$

其中 2_a 和 2_b 的关键字相同。

经过快速排序的划分，它们的相对位置可能发生改变，因此快速排序不保证稳定性。

2. 是否原地排序?

通常认为快速排序是原地排序算法，因为划分直接在原数组中进行，不需要额外的线性规模辅助数组。

但它仍然需要递归栈空间。

3. 是否属于比较排序?

属于。

快速排序通过比较元素的关键字来完成排序。

八、快速排序与其他排序算法对比

排序算法	最好时间复杂度	平均时间复杂度	最坏时间复杂度	辅助空间	稳定性
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
折半插入排序	$O(n\log n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
冒泡排序（优化后）	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
希尔排序	与增量序列有关	与增量序列有关	与增量序列有关	$O(1)$	不稳定
快速排序	$O(n\log n)$	$O(n\log n)$	$O(n^2)$	平均 $O(\log n)$	**不稳定**
堆排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(1)$	不稳定
归并排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	通常 $O(n)$	稳定

注：折半插入排序的最好时间复杂度为 $O(n\log n)$，是指标准实现每次都执行折半查找的情况。它虽然减少了关键字比较次数，但元素移动仍可能需要 $O(n^2)$ 时间。

九、408 高频考点

考点 1：快速排序的枢轴位置

一次划分结束后，枢轴已经处于最终排序位置。

但枢轴两侧的子序列不一定有序。

考点 2：快速排序的递归次数与递归深度

不要混淆递归调用次数和递归深度。

- 划分均衡时，递归深度通常为 $O(\log n)$。
- 划分极不均衡时，递归深度可能达到 $O(n)$。

考点 3：快速排序的最坏情况

如果每次都选择第一个元素作为枢轴，而输入序列已经有序或逆序，就可能导致每次划分都极不均衡。

此时：

$$\boxed{T(n)=O(n^2)}$$

随机选择枢轴或采用更合理的枢轴选择策略，可以降低出现极端划分的风险，但不能改变普通快速排序在最坏情况下的 $O(n^2)$ 时间复杂度上界。

考点 4：快速排序的稳定性

快速排序不稳定。

不能因为枢轴两侧满足大小关系，就认为相同关键字的元素仍然保持原来的相对顺序。

考点 5：快速排序与归并排序

两者都是分治思想的典型应用，但实现方式不同。

- **快速排序：** 先划分，再递归排序左右子序列。
- **归并排序：** 先递归排序左右子序列，再将两个有序子序列合并。

因此，快速排序的主要工作在划分阶段，而归并排序的主要合并工作发生在递归返回阶段。

十、考前速记

问题	结论
基本思想	分治法
核心操作	划分（Partition）
一次划分后，谁的位置确定?	枢轴元素
最好时间复杂度	$O(n \log n)$
平均时间复杂度	$O(n \log n)$
最坏时间复杂度	$O(n^2)$
平均辅助空间复杂度	$O(\log n)$
最坏辅助空间复杂度	$O(n)$
稳定性	不稳定
是否原地排序	通常认为是
有序序列作为输入一定是最坏情况吗?	不一定，取决于枢轴选择策略
快速排序的主要优势	平均效率高，原地划分
快速排序的主要缺点	最坏时间复杂度高，递归深度可能很大

最后记住这四句话

1. **快速排序的核心是划分，枢轴在一次划分后到达最终位置。**
2. **最好、平均时间复杂度为 $O(n \log n)$，最坏为 $O(n^2)$。**
3. **快速排序是不稳定的原地排序算法，但需要递归栈空间。**
4. **划分越均衡，递归深度通常越小，算法效率通常越高。**

我大致描述一下快速排序的过程：

1. 一般取待排序序列的第一个元素作为枢轴元素，先把这个枢轴元素暂存到另一个缓冲区里。
2. 此时它的左侧会出现一个空位置。接下来从右边开始找，找比枢轴元素小的数，把它放到左边的空位置上去。
3. 至此，左边的空位置被填了，右边又出现一个新的空位置。此时再从左边开始找，找比枢轴元素大的数，放到右边的空位置去。
4. 循环往复，直到指向low和high的两个指针碰到一起，就停止操作。
5. 最后把暂存的枢轴元素放到中间的空位置里，就完成了快速排序。

你这个理解已经非常接近正确了！👏 尤其是你抓住了**「挖坑法」的核心：填上一个空位，就会在另一侧产生新的空位，左右交替填坑**。这说明你已经理解了快速排序一次划分的基本过程。

不过，你的描述里有一个关键细节需要修正：**第 5 步完成的不是整个快速排序，而是一次划分（Partition）。**

我们逐条看一下。

一、逐条检查你的理解

步骤	你的理解	判断	补充说明
1	取第一个元素作为枢轴，暂存到缓冲区	✅	通常暂存在局部变量 `pivot` 中，不需要专门开辟数组
2	从右边寻找比枢轴小的元素，填入左边空位	✅	更准确地说，寻找小于枢轴的元素
3	从左边寻找比枢轴大的元素，填入右边空位	✅	更准确地说，寻找大于枢轴的元素
4	`low` 和 `high` 相遇后停止	✅	当两个指针相遇时，本次划分结束
5	将枢轴放入空位，完成快速排序	⚠️	这里只完成了一次划分，还需要递归排序左右子序列

特别注意第 5 步

假设原序列为：

$$[49, 38, 65, 97, 76, 13, 27]$$

经过一次划分后：

$$[27, 38, 13, \boxed{49}, 76, 97, 65]$$

此时只能确定：

- `49` 左侧的元素都不大于 `49`。
- `49` 右侧的元素都不小于 `49`。
- `49` 已经处于最终排序位置。

但是，左边的 `[27,38,13]` 和右边的 `[76,97,65]` 内部仍然无序。

因此，还要继续分别对左右子序列进行快速排序。

整个过程可以概括为：

```
快速排序
   |
   ▼
 一次划分
   |
   ├──── 枢轴左侧 → 递归快速排序
   |
   └──── 枢轴右侧 → 递归快速排序
                      |
                      ▼
                  最终有序
```

二、我帮你整理成更准确的背诵版本

下面这段可以直接用作你的考研笔记：

> **快速排序的基本过程：**
>
> 1. 通常选择待排序区间的第一个元素作为枢轴，将其暂存在变量 `pivot` 中，使原位置成为一个空位。
> 2. 从右向左扫描，寻找小于枢轴的元素，将其放入左侧空位。
> 3. 此时右侧产生新的空位，再从左向右扫描，寻找大于枢轴的元素，将其放入右侧空位。
> 4. 重复上述过程，使 `low` 和 `high` 交替移动，直到两个指针相遇。
> 5. 将暂存的枢轴放入指针相遇的位置，完成一次划分。此时枢轴已处于最终排序位置。
> 6. 分别对枢轴左侧和右侧的子序列递归执行快速排序，直至子序列长度不超过 1。

三、再帮你提炼一个最重要的区别

挖坑、填坑、枢轴归位，是一次划分；递归处理左右子序列，才是完整的快速排序。

你可以把快速排序理解成两个层次：

- **第一层：** 怎样把一个序列划分成枢轴左边和右边两部分？
- **第二层：** 怎样让这两部分也分别变得有序？

第一层是 Partition() 函数，第二层是 QuickSort() 递归函数。

这两个层次区分清楚之后，你再看快速排序的 C 语言代码，就会容易理解得多。

给我简单选择排序的笔记。

> **正在搜索网页**
>
> 思考了 6s

简单选择排序（Simple Selection Sort）笔记

一、基本思想

简单选择排序：每一趟从待排序区间中选出最小的元素，将其与待排序区间的第一个元素交换。

排序过程分为两个部分：

- **有序区：** 已经排好序的部分，位于数组左侧。
- **无序区：** 尚未排好序的部分，位于数组右侧。

每一趟排序后，有序区增加一个元素，无序区减少一个元素。

二、排序过程示例

假设待排序序列为：

$$[49,\ 38,\ 65,\ 97,\ 76,\ 13,\ 27]$$

第 1 趟

在整个序列中寻找最小值 `13`，与第一个元素 `49` 交换：

$$[13,\ 38,\ 65,\ 97,\ 76,\ 49,\ 27]$$

此时，`13` 已经排好序。

第 2 趟

在剩余无序区间中寻找最小值 `27`，与第二个元素 `38` 交换：

$$[13,\ 27,\ 65,\ 97,\ 76,\ 49,\ 38]$$

此时，`13`、`27` 已经排好序。

第 3 趟

在剩余无序区间中寻找最小值 `38`，与 `65` 交换：

$$[13,\ 27,\ 38,\ 97,\ 76,\ 49,\ 65]$$

继续进行，直到整个序列有序。

理解重点：每一趟都在无序区中寻找最小值，而不是只比较相邻元素。

三、交互式演示

可以通过下面的交互演示，观察每一趟如何寻找最小值、记录最小值的位置，并将其放到无序区的最前面。

≣genui≣{"learning_viz":{"type_id":"SELECTION_SORT","initial_values":{"order":"6,3,8,2,7,1,5,4"},"locale_override":"zh-CN"}}≣

四、C 语言实现

```
void SelectSort(int A[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int min = i;  // 记录最小元素的位置

        for (int j = i + 1; j < n; j++)
        {
            if (A[j] < A[min])
                min = j;
        }

        // 将最小元素与无序区第一个元素交换
        if (min != i)
        {
            int temp = A[i];
            A[i] = A[min];
            A[min] = temp;
        }
    }
}
```

代码理解

代码	作用
`i`	指向无序区的第一个位置
`min = i`	假设无序区第一个元素最小
`j = i + 1`	从无序区第二个元素开始比较
`A[j] < A[min]`	发现更小的元素，更新最小值位置
交换 `A[i]` 和 `A[min]`	将本趟最小元素放到有序区末尾

注意：`min` 记录的是**数组下标**，不是最小元素的值。

五、复杂度与稳定性

性质	结论
最好时间复杂度	$O(n^2)$
平均时间复杂度	$O(n^2)$
最坏时间复杂度	$O(n^2)$
空间复杂度	$O(1)$
稳定性	**不稳定**
排序方式	原地排序

为什么时间复杂度始终是 $O(n^2)$?

无论原序列是否有序，每一趟都需要扫描剩余无序区，寻找最小元素。

比较次数为：

$$(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2}$$

因此，最好、平均和最坏情况下的时间复杂度均为：

$$\boxed{O(n^2)}$$

为什么不稳定?

例如：

$$[5_a,\ 5_b,\ 1]$$

第一趟选择最小值 1，与第一个元素 5_a 交换：

$$[1,\ 5_b,\ 5_a]$$

两个关键字相同的元素，原本的相对顺序是 5_a 在前、5_b 在后，交换后却变成 5_b 在前、5_a 在后。

因此，简单选择排序**不稳定**。

六、与冒泡排序的区别

对比项	简单选择排序	冒泡排序
核心操作	选择最小元素并交换	比较相邻元素并交换
每趟结果	无序区最小元素归位	无序区最大元素归位
比较次数	始终为 $\Theta(n^2)$	通常为 $\Theta(n^2)$
交换次数	最多 $n-1$ 次	最坏为 $\Theta(n^2)$
稳定性	不稳定	通常稳定

七、408 考前速记

记住以下五点即可：

1. **核心思想：** 每趟从无序区选出最小元素。
2. **排序过程：** 将最小元素与无序区第一个元素交换。
3. **时间复杂度：** 最好、平均、最坏均为 $O(n^2)$。
4. **空间复杂度：** $O(1)$，属于原地排序。
5. **稳定性：** 不稳定，因为交换可能改变相同关键字元素的相对顺序。

一句话记忆：选择排序找最小，冒泡排序推最大；选择排序交换少，但不保证稳定。

给我堆排序的笔记。

堆排序（Heap Sort）笔记

堆排序是 408 数据结构排序部分的重点内容，尤其要掌握 **堆的定义、建堆过程、调整堆的过程、堆排序的执行步骤，以及时间复杂度和稳定性**。

一、基本思想

堆排序是一种利用堆这种数据结构进行排序的算法。

首先，需要将待排序序列构造成一个堆，然后反复取出堆顶元素，并调整剩余元素，使其重新满足堆的性质，最终完成排序。

1. 什么是堆?

堆是一棵满足特定大小关系的完全二叉树。

堆分为两种：

- **大根堆（最大堆）：** 每个非叶结点的值都不小于其左右孩子的值。
- **小根堆（最小堆）：** 每个非叶结点的值都不大于其左右孩子的值。

大根堆满足：

$$A[i] \geq A[2i], \qquad A[i] \geq A[2i+1]$$

小根堆满足：

$$A[i] \leq A[2i], \qquad A[i] \leq A[2i+1]$$

以上下标关系采用**从 1 开始的数组下标**，并且只对存在的孩子成立。

2. 大根堆与小根堆的区别

≣genui≣{"learning_viz":{"type_id":"SELECTION_SORT","initial_values":{"order":"6,3,8,2,7,1,5,4"},"locale_override":"zh-CN"}}≣

上面的交互演示是选择排序，并非堆排序，因此不适合用来观察堆结构。理解堆时，应重点关注下面的结构示意图。

大根堆示例：

```
flowchart TD
    A["90"] --> B["70"]
    A --> C["80"]
```

```
    B --> D["20"]
    B --> E["60"]
    C --> F["50"]
    C --> G["30"]
```

每个父结点都不小于自己的孩子，因此根结点 `90` 是整个堆中的最大值。

小根堆示例：

```
flowchart TD
    A["10"] --> B["30"]
    A --> C["20"]
    B --> D["70"]
    B --> E["60"]
    C --> F["50"]
    C --> G["80"]
```

每个父结点都不大于自己的孩子，因此根结点 `10` 是整个堆中的最小值。

注意：**堆只要求父结点与孩子之间满足大小关系，不要求同一层或左右子树内部有序。**

二、堆排序的基本流程

以升序排序为例，通常使用大根堆。

整个过程分为三个阶段：

1. **建立大根堆：** 将待排序序列调整为大根堆，使最大元素位于堆顶。
2. **交换堆顶与末尾元素：** 将当前最大元素放到数组末尾，确定其最终位置。
3. **调整剩余堆：** 将堆顶元素向下调整，使剩余元素重新满足大根堆性质，然后继续交换。

重复上述操作，直到堆中只剩一个元素。

为什么升序排序使用大根堆?

大根堆的堆顶是当前最大元素。

每次把堆顶元素与当前堆的最后一个元素交换，就能将最大元素放到数组末尾。

随后缩小堆的范围，再对剩余元素进行调整。

因此，最大元素依次从右向左确定位置，最终得到升序序列。

同理，**使用小根堆，可以依次将最小元素放到数组末尾，最终得到降序序列。**

三、调整堆（核心操作）

堆排序最重要的操作是**向下调整（筛选，Sift Down）**。

假设某个结点的左右子树已经分别满足大根堆的性质，但该结点本身可能比孩子小，就可以通过向下调整恢复大根堆。

1. 向下调整的过程

以某个结点为例：

```
flowchart TD
    A["当前结点 40"] --> B["左孩子 70"]
    A --> C["右孩子 60"]
    B --> D["子树中的其他结点"]
    C --> E["子树中的其他结点"]
```

当前结点是 `40`，左孩子是 `70`，右孩子是 `60`。

调整步骤：

1. 比较当前结点与左右孩子。
2. 找出三个结点中的最大值。
3. 如果最大值是当前结点，调整结束。
4. 如果最大值是某个孩子，就交换当前结点与该孩子。
5. 继续向下检查交换后的位置，直到满足大根堆性质，或者到达叶结点。

例如：

初始状态：

```
    40
   /  \
  70   60
```

交换 `40` 和 `70`：

```
    70
   /  \
  40   60
```

此时，如果 `40` 的孩子也满足堆的性质，那么调整结束。

注意：向下调整时，应优先与较大的孩子比较和交换。 如果与较小的孩子交换，父结点可能仍然小于另一个孩子，无法保证大根堆性质。

2. 向下调整的 C 语言实现

以下采用数组下标从 `1` 开始的写法：

```
void SiftDown(int A[], int k, int n)
{
    int temp = A[k];

    for (int child = 2 * k; child <= n; child *= 2)
    {
        // 选择左右孩子中较大的一个
        if (child < n && A[child] < A[child + 1])
            child++;

        // 当前元素已经不小于较大的孩子
        if (temp >= A[child])
            break;

        // 将较大的孩子向上移动
        A[k] = A[child];
        k = child;
    }

    A[k] = temp;
}
```

其中：

- `k`：待调整结点的下标。
- `n`：当前堆的最后一个有效下标。
- `child`：较大孩子的下标。
- `temp`：暂存待调整结点的值。

这段代码使用了**暂存元素、将较大孩子逐层上移、最后放回暂存元素**的方法，不一定需要每次都进行完整交换。

四、建立大根堆

建堆的关键是：**从最后一个非叶结点开始，依次向前进行向下调整。**

对于有 `n` 个元素的完全二叉树：

- 叶结点下标为：

$$\left\lfloor \frac{n}{2} \right\rfloor + 1$$

到 n。

- 最后一个非叶结点下标为：

$$\left\lfloor \frac{n}{2} \right\rfloor$$

因此，建堆代码为：

```
void BuildMaxHeap(int A[], int n)
{
    for (int i = n / 2; i >= 1; i--)
        SiftDown(A, i, n);
}
```

为什么必须从后往前调整？

因为向下调整一个结点时，需要保证它的左右子树已经满足堆的性质。

从最后一个非叶结点开始向前处理，可以保证每次调整时，该结点的左右子树都已经满足大根堆的要求。

建堆示例

假设初始序列为：

$$[4, 6, 8, 5, 9]$$

下标从 1 开始：

下标	1	2	3	4	5
元素	4	6	8	5	9

最后一个非叶结点为：

$$\left\lfloor \frac{5}{2} \right\rfloor = 2$$

因此，依次调整下标为 2 和 1 的结点。

第一步：调整下标为 2 的结点。

结点 6 的孩子是 5 和 9。

最大值是 9，交换后得到：

$$[4, 9, 8, 5, 6]$$

第二步：调整下标为 1 的结点。

结点 4 的孩子是 9 和 8。

先将 4 与较大的孩子 9 调整，再检查新的位置。

最终得到大根堆：

$$\boxed{[9, 6, 8, 5, 4]}$$

可以验证：

- 9 不小于 6 和 8。
- 6 不小于 5 和 4。

因此，建堆完成。

建堆复杂度

建堆的时间复杂度为：

$$\boxed{O(n)}$$

这是 408 的高频考点：建堆不是 $O(n \log n)$，而是 $O(n)$。

原因是，大多数结点距离叶结点很近，只有少数结点需要进行较多层的调整，因此所有结点的调整代价加起来为线性量级。

五、堆排序的完整 C 语言代码

以下代码采用数组下标从 1 开始的方式，`A[0]` 不参与排序。

```
void SiftDown(int A[], int k, int n)
{
    int temp = A[k];

    for (int child = 2 * k; child <= n; child *= 2)
    {
        if (child < n && A[child] < A[child + 1])
            child++;

        if (temp >= A[child])
            break;

        A[k] = A[child];
        k = child;
    }

    A[k] = temp;
}

void BuildMaxHeap(int A[], int n)
{
    for (int i = n / 2; i >= 1; i--)
```

```
        SiftDown(A, i, n);
}

void HeapSort(int A[], int n)
{
    BuildMaxHeap(A, n);

    for (int i = n; i > 1; i--)
    {
        // 将当前最大元素放到数组末尾
        int temp = A[1];
        A[1] = A[i];
        A[i] = temp;

        // 剩余 i - 1 个元素重新调整为大根堆
        SiftDown(A, 1, i - 1);
    }
}
```

代码逻辑

```
HeapSort
   |
   ▼
BuildMaxHeap
   |
   ▼
建立大根堆
   |
   ▼
交换堆顶与末尾元素
   |
   ▼
堆的有效范围缩小 1
   |
   ▼
SiftDown(1, 剩余元素个数)
   |
   ▼
重复，直到只剩 1 个元素
```

注意：建堆和排序阶段使用的堆大小不同。

- 建堆时，堆大小是 `n`。
- 每次将最大元素放到末尾后，堆大小减为 `i - 1`。

- 已经排好序的后缀不再参与调整。

六、堆排序的复杂度与稳定性

性质	结论
最好时间复杂度	$O(n \log n)$
平均时间复杂度	$O(n \log n)$
最坏时间复杂度	$O(n \log n)$
建堆时间复杂度	$O(n)$
空间复杂度	$O(1)$
稳定性	不稳定
排序方式	原地排序

为什么时间复杂度是 $O(n \log n)$?

堆排序主要包含两个阶段：

阶段一：建堆

$$O(n)$$

阶段二：反复取出堆顶并调整

总共进行约 $n-1$ 次，每次向下调整的代价最多为：

$$O(\log n)$$

因此：

$$O(n) + O(n \log n) = \boxed{O(n \log n)}$$

为什么堆排序不稳定?

堆排序会交换堆顶元素和当前堆的末尾元素。

这种交换可能改变两个相同关键字元素的相对次序，因此堆排序不保证稳定性。

七、堆排序与简单选择排序的区别

两种算法都可以理解为反复选出当前最大或最小的元素，但它们寻找元素的方式不同。

对比项	简单选择排序	堆排序
核心思想	每趟遍历无序区寻找最小元素	利用堆顶快速获取最大元素
每次选取元素的代价	需要线性扫描无序区	取堆顶为 $O(1)$，随后调整为 $O(\log n)$
总时间复杂度	$O(n^2)$	$O(n \log n)$
辅助空间	$O(1)$	$O(1)$
稳定性	不稳定	不稳定

核心区别：简单选择排序每次重新扫描无序区，而堆排序通过维护堆结构，减少了寻找极值的代价。

八、408 考前速记

建议优先记住以下内容：

1. **大根堆：** 父结点不小于孩子；堆顶是最大值。
2. **升序排序：** 通常使用大根堆，每次将堆顶与当前堆末尾元素交换。
3. **建堆顺序：** 从最后一个非叶结点开始，向前依次向下调整。
4. **最后一个非叶结点：**

$$\boxed{\left\lfloor \frac{n}{2} \right\rfloor}$$

5. **建堆时间复杂度：** $O(n)$。
6. **堆排序总时间复杂度：** 最好、平均、最坏均为 $O(n \log n)$。
7. **空间复杂度：** $O(1)$。
8. **稳定性：** 不稳定。

最后用一句话理解堆排序

先建堆，再反复将堆顶最大值放到末尾，并缩小堆的范围、重新调整堆，直到整个序列有序。

给我归并排序的笔记。

归并排序（Merge Sort）笔记

归并排序是 408 数据结构排序部分的重要算法。重点掌握：**分治思想、二路归并过程、递归实现、时间复杂度、空间复杂度和稳定性**。

一、基本思想

归并排序采用**分治法**，将待排序序列不断划分为更小的子序列，分别排序后，再将有序子序列合并成一个更大的有序序列。

以常见的二路归并排序为例，主要分为三个步骤：

1. **分解：** 将待排序序列分成左右两个子序列。
2. **解决：** 递归地对左右子序列进行排序，直到每个子序列只剩一个元素。
3. **合并：** 将两个已经有序的子序列合并成一个有序序列。

其中，单个元素天然有序，因此可以作为递归的终止条件。

记忆重点：快速排序是先划分，再递归；归并排序是先递归，再合并。

二、归并排序的执行过程

假设待排序序列为：

$$[38,\ 12,\ 27,\ 43,\ 9,\ 31,\ 18,\ 25]$$

归并排序会先不断拆分，再逐层合并。

1. 拆分阶段

不断将序列分成左右两部分，直到每个子序列只剩一个元素。

```
flowchart TD
    A["38 12 27 43 9 31 18 25"]
    A --> B["38 12 27 43"]
    A --> C["9 31 18 25"]
    B --> D["38 12"]
    B --> E["27 43"]
    C --> F["9 31"]
    C --> G["18 25"]
    D --> H["38"]
    D --> I["12"]
```

```
    E --> J["27"]
    E --> K["43"]
    F --> L["9"]
    F --> M["31"]
    G --> N["18"]
    G --> O["25"]
```

拆分结束后，每个子序列都只有一个元素，自然有序。

2. 合并阶段

从最小的子序列开始，两两合并。

例如：

$$[38] + [12] \longrightarrow [12, 38]$$

$$[27] + [43] \longrightarrow [27, 43]$$

然后继续合并：

$$[12, 38] + [27, 43] \longrightarrow [12, 27, 38, 43]$$

另一半同理：

$$[9, 31] + [18, 25] \longrightarrow [9, 18, 25, 31]$$

最后，将两个有序子序列合并：

$$\begin{aligned} &[12, 27, 38, 43] \\ +&[9, 18, 25, 31] \\ \longrightarrow\ &[9, 12, 18, 25, 27, 31, 38, 43] \end{aligned}$$

最终得到有序序列。

3. 交互式观察拆分与合并

可以拖动步骤，观察归并排序如何将序列拆成单个元素，再逐层合并为有序序列。

≣genui≣{"learning_viz":{"type_id":"MERGE_SORT","initial_values":{"value1":38,"value2":12,"value3":27,"value4":43,"value5":9,"value6":31,"value7":18,"value8":25},"locale_override":"zh-CN"}}≣

三、核心操作：二路归并

二路归并（Merge）指将两个已经有序的子序列合并成一个有序序列。

例如，现有两个有序序列：

$$L = [12, 27, 38, 43]$$

$$R = [9, 18, 25, 31]$$

归并时，分别使用两个指针指向左右序列当前尚未处理的第一个元素，每次比较这两个元素，将较小者放入结果序列，然后移动对应指针。

归并过程

步骤	比较	取出的元素	当前结果
1	12 与 9	9	9
2	12 与 18	12	9, 12
3	27 与 18	18	9, 12, 18
4	27 与 25	25	9, 12, 18, 25
5	27 与 31	27	9, 12, 18, 25, 27
6	38 与 31	31	9, 12, 18, 25, 27, 31

此时右侧序列已经处理完毕，将左侧剩余的 `38`、`43` 直接追加到结果末尾。

最终得到：

$$\boxed{[9, 12, 18, 25, 27, 31, 38, 43]}$$

二路归并的注意事项

1. 每次比较两个子序列当前的第一个未处理元素。
2. 将较小元素放入结果序列，并移动对应指针。
3. 如果其中一个子序列处理完毕，将另一个子序列的剩余元素直接追加。
4. **每个元素只需要被处理一次，因此，合并长度为 n 的两个有序子序列的总时间复杂度为 $O(n)$。**

四、C 语言实现

以下代码采用从 `0` 开始的数组下标，并使用辅助数组 `B` 存放归并结果。

1. 二路归并函数

```
void Merge(int A[], int B[], int low, int mid, int high)
{
    int i = low;
```

```
    int j = mid + 1;
    int k = low;

    // 比较两个有序子序列，依次取出较小元素
    while (i <= mid && j <= high)
    {
        if (A[i] <= A[j])
            B[k++] = A[i++];
        else
            B[k++] = A[j++];
    }

    // 复制左侧剩余元素
    while (i <= mid)
        B[k++] = A[i++];

    // 复制右侧剩余元素
    while (j <= high)
        B[k++] = A[j++];

    // 将归并结果复制回原数组
    for (int t = low; t <= high; t++)
        A[t] = B[t];
}
```

这里使用：

```
if (A[i] <= A[j])
```

而不是：

```
if (A[i] < A[j])
```

原因是：当两个元素的关键字相等时，优先取左侧子序列的元素，可以保持相同关键字元素的原有相对顺序，这是归并排序具有稳定性的关键之一。

2. 递归归并排序

```
void MergeSort(int A[], int B[], int low, int high)
{
    if (low < high)
    {
        int mid = low + (high - low) / 2;

        // 递归排序左侧子序列
```

```
        MergeSort(A, B, low, mid);

        // 递归排序右侧子序列
        MergeSort(A, B, mid + 1, high);

        // 合并两个有序子序列
        Merge(A, B, low, mid, high);
    }
}
```

调用方式：

```
int B[n];
MergeSort(A, B, 0, n - 1);
```

这里的 `B` 是辅助数组，实际项目中可以将其作为动态分配的辅助空间。

3. 代码逻辑总结

```
MergeSort(A, low, high)
        │
        ▼
    low < high ?
        │
        ▼
计算中间位置 mid
        │
        ├──── 递归排序左半部分
        │
        ├──── 递归排序右半部分
        │
        └──── Merge 合并左右有序序列
```

代码易错点： `Merge()` 负责合并两个已经有序的子序列；`MergeSort()` 负责递归排序并调用 `Merge()`。二者的职责不同。

五、时间复杂度分析

归并排序每次将序列分成两个子序列，分别递归排序，再用线性时间合并。

其递推关系为：

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

其中：

- $2T(n/2)$：递归排序两个子序列。
- $O(n)$：合并两个有序子序列。

每次拆分后，子序列规模约减半，因此递归层数约为：

$$\log_2 n$$

每一层所有合并操作的总时间复杂度为 $O(n)$，因此：

$$\boxed{T(n) = O(n \log n)}$$

归并排序的最好、平均和最坏时间复杂度均为：

$$\boxed{O(n \log n)}$$

注意：即使原序列已经有序，标准递归归并排序仍然会执行拆分和合并，因此时间复杂度仍为 $O(n \log n)$。

六、空间复杂度

标准的递归二路归并排序需要额外的辅助数组，用于存放合并结果。

- 辅助数组空间：$O(n)$。
- 递归调用栈：通常为 $O(\log n)$。

因此，总辅助空间复杂度为：

$$\boxed{O(n)}$$

因为 $O(n)$ 大于 $O(\log n)$，所以辅助数组占据主要空间开销。

七、稳定性

标准二路归并排序是稳定的排序算法。

稳定性是指：如果两个元素的关键字相同，排序后它们的相对顺序仍然不变。

例如：

$$[3_a, 1, 3_b]$$

其中 3_a 和 3_b 的关键字相同，且 3_a 原本在 3_b 前面。

稳定排序后应当是：

$$[1, 3_a, 3_b]$$

标准归并排序在合并时，遇到相同关键字优先取左侧元素，就可以保持这种相对顺序。

注意，稳定性与具体实现有关；采用其他处理方式时，不能不加判断地认定其稳定。

八、归并排序与快速排序的区别

对比项	归并排序	快速排序
基本思想	分治法	分治法
核心操作	合并两个有序子序列	划分序列
递归顺序	先递归，再合并	先划分，再递归
最好时间复杂度	$O(n \log n)$	$O(n \log n)$
平均时间复杂度	$O(n \log n)$	$O(n \log n)$
最坏时间复杂度	$O(n \log n)$	$O(n^2)$
辅助空间复杂度	$O(n)$	平均 $O(\log n)$
稳定性	稳定	不稳定

记忆方法：

- 归并排序：先拆分，后合并；合并时比较两个有序子序列。
- 快速排序：先划分，后递归；划分时确定枢轴的位置。

九、408 考前速记

考点	结论
基本思想	分治法
核心操作	二路归并
递归终止条件	子序列长度不超过 1
合并两个有序序列	$O(n)$
最好时间复杂度	$O(n\log n)$
平均时间复杂度	$O(n\log n)$
最坏时间复杂度	$O(n\log n)$
辅助空间复杂度	$O(n)$
稳定性	稳定
是否原地排序	标准实现不是严格的原地排序

最后记住这五点

1. **归并排序采用分治法，先递归排序，再合并。**
2. 二路归并要求输入的两个子序列已经有序。
3. 合并两个总长度为 n 的有序子序列，时间复杂度为 $O(n)$。
4. 标准归并排序的最好、平均、最坏时间复杂度均为 $O(n\log n)$。
5. 标准归并排序需要 $O(n)$ 辅助空间，并且是稳定排序。

一句话总结：归并排序就是不断拆分，直到每个子序列只有一个元素，再通过二路归并逐层合并成完整的有序序列。

给我计数排序的笔记。

计数排序（Counting Sort）笔记

计数排序是 408 排序部分中需要了解的**非比较排序算法**。它与快速排序、归并排序等算法不同，不是通过反复比较元素大小来确定顺序，而是统计每个关键字出现的次数，进而确定元素的位置。

一、基本思想

计数排序：统计每个关键字出现的次数，再根据关键字的大小关系，将元素放到有序位置。

假设待排序序列为：

$$A = [4, 2, 2, 8, 3, 3, 1]$$

首先统计每个数出现的次数：

数值	出现次数
1	1
2	2
3	2
4	1
5	0
6	0
7	0
8	1

然后按照数值从小到大的顺序，依次输出对应次数的元素。

最终得到：

$$\boxed{[1, 2, 2, 3, 3, 4, 8]}$$

核心区别：比较排序依靠比较元素来排序，计数排序依靠统计关键字的出现情况来排序。

二、计数排序的执行过程

仍然使用序列：

$$A = [4, 2, 2, 8, 3, 3, 1]$$

第一步：统计出现次数

假设最小值为 `1`，最大值为 `8`，创建一个计数数组 `count`，其中 `count[i]` 表示数值 `i` 出现的次数。

统计后：

下标（数值）	1	2	3	4	5	6	7	8
`count[i]`	1	2	2	1	0	0	0	1

第二步：根据计数结果输出

从小到大遍历计数数组：

- `1` 出现 1 次，输出一个 `1`。
- `2` 出现 2 次，输出两个 `2`。
- `3` 出现 2 次，输出两个 `3`。
- `4` 出现 1 次，输出一个 `4`。
- `5`、`6`、`7` 均未出现，不输出。
- `8` 出现 1 次，输出一个 `8`。

最终得到：

$$[1, 2, 2, 3, 3, 4, 8]$$

这种方法简单直观，但有一个限制：**它要求关键字能够映射到一个可行的计数数组下标，并且关键字的取值范围不能过大。**

三、C 语言实现：基础计数排序

以下实现适用于**非负整数**，并假设最大值不会导致计数数组过大。

```
void CountingSort(int A[], int n, int maxValue)
{
    int count[maxValue + 1];

    // 初始化计数数组
    for (int i = 0; i <= maxValue; i++)
        count[i] = 0;

    // 统计每个数值出现的次数
```

```
    for (int i = 0; i < n; i++)
        count[A[i]]++;

    // 按照数值从小到大重新写入原数组
    int k = 0;

    for (int i = 0; i <= maxValue; i++)
    {
        while (count[i] > 0)
        {
            A[k++] = i;
            count[i]--;
        }
    }
}
```

例如：

```
int A[] = {4, 2, 2, 8, 3, 3, 1};
CountingSort(A, 7, 8);
```

排序结果为：

```
1 2 2 3 3 4 8
```

代码理解

代码	作用
`count[i] = 0`	初始化计数数组
`count[A[i]]++`	统计元素出现次数
外层 `for`	按关键字从小到大遍历
`while (count[i] > 0)`	根据出现次数输出元素

注意：这个基础版本会直接将数值写回数组，适用于整数关键字；如果元素除了关键字之外还包含其他信息，就需要考虑如何保留这些信息。

四、计数排序的时间复杂度

设：

- n：待排序元素的个数。
- k：关键字的取值范围大小。

对于非负整数，若关键字范围为 0 到 m，则：

$$k = m + 1$$

计数排序主要执行三项工作：

1. 初始化计数数组：$O(k)$。
2. 统计元素出现次数：$O(n)$。
3. 遍历计数数组并输出：$O(n + k)$。

因此，总时间复杂度为：

$$\boxed{O(n + k)}$$

如果关键字取值范围与元素数量处于相同数量级，即 $k = O(n)$，则：

$$\boxed{O(n)}$$

但如果关键字范围远大于元素数量，初始化和遍历计数数组的开销就可能非常大。

空间复杂度

基础实现需要一个长度为 k 的计数数组，因此：

$$\boxed{O(k)}$$

如果采用额外输出数组的稳定计数排序实现，辅助空间通常为：

$$\boxed{O(n + k)}$$

五、计数排序是否稳定?

计数排序不一定稳定，取决于具体实现。

1. 基础计数排序

上面直接统计每个数值的出现次数，再按数值输出。

如果待排序对象除了关键字外还带有其他信息，这种实现没有记录相同关键字元素的原始顺序，因此不能保证稳定性。

2. 稳定计数排序

要实现稳定计数排序，通常需要：

1. 统计每个关键字出现的次数。
2. 将次数转换为累计计数，确定每个关键字在结果数组中的结束位置。
3. 从原数组的右向左遍历元素。
4. 根据累计计数，将元素放入输出数组的对应位置，并更新计数。
5. 将输出数组复制回原数组（如有需要）。

这种实现可以保证相同关键字元素的相对顺序不变。

408 记忆重点：计数排序的稳定性取决于实现，基础计数版本不能直接认定为稳定。

六、计数排序的优缺点

优点

- 当关键字范围较小时，排序效率高。
- 在适用条件下，可以达到 $O(n+k)$ 的时间复杂度。
- 不依赖元素之间的比较。

缺点

- 需要额外空间保存计数信息。
- 关键字范围过大时，空间和时间开销可能很高。
- 基础实现主要适用于整数或能够映射到有限整数范围的关键字。
- 基础实现不保证稳定性。

七、计数排序与比较排序的区别

对比项	计数排序	快速排序	归并排序
排序类型	非比较排序	比较排序	比较排序
核心操作	统计关键字出现次数	划分	合并
时间复杂度	$O(n+k)$	平均 $O(n\log n)$	$O(n\log n)$
辅助空间	基础实现为 $O(k)$	平均递归栈 $O(\log n)$	通常 $O(n)$
稳定性	取决于实现	不稳定	标准实现稳定
适用条件	关键字范围较小	一般可比较关键字	一般可比较关键字

八、408 考前速记

考点	结论
基本思想	统计每个关键字出现的次数
排序类型	非比较排序
时间复杂度	$O(n+k)$
基础实现的空间复杂度	$O(k)$
稳定性	取决于实现
主要优势	关键字范围较小时效率高
主要限制	关键字范围过大时开销较大

一句话记忆

计数排序不靠比较大小，而是统计每个关键字出现的次数，再按关键字顺序输出；时间复杂度与元素数量和关键字范围有关。

对上面我所提问的所有算法进行归纳比较。

排序算法总复习：归纳、比较与考研重点

把前面学过的排序算法放在一起比较，最重要的不是死记每个算法的代码，而是建立一张完整的知识地图：

1. **时间复杂度**：最好、平均、最坏分别是多少？
2. **空间复杂度**：是否原地排序？是否需要辅助数组？
3. **稳定性**：相同关键字的元素，排序后相对次序是否保持不变？
4. **基本思想**：插入、交换、选择、归并，还是利用关键字的分布特征？
5. **关键细节**：哪些算法最好情况下能达到 $O(n)$？哪些算法最坏情况下仍是 $O(n\log n)$？哪些算法不属于比较排序？

我们前面涉及的算法包括：

- 直接插入排序
- 折半插入排序
- 希尔排序
- 冒泡排序
- 快速排序
- 简单选择排序
- 堆排序
- 二路归并排序
- 计数排序

下面把它们整理成一份适合考研复习和放入 Obsidian 的对照笔记。

一、九种排序算法总表

排序算法	最好时间复杂度	平均时间复杂度	最坏时间复杂度	辅助空间复杂度	稳定性
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
折半插入排序	$O(n\log n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
希尔排序	取决于增量序列	取决于增量序列	取决于增量序列	$O(1)$	不稳定
冒泡排序（带标志优化）	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
快速排序	$O(n\log n)$	$O(n\log n)$	$O(n^2)$	平均 $O(\log n)$	不稳定
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(1)$*	不稳定
二路归并排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(n)$	稳定
计数排序	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$*	取决于实现

表中符号说明：

- n：待排序元素的数量。
- k：关键字的取值范围大小。例如，待排序整数的取值范围为 0 ~ 999，则 k = 1000。
- 希尔排序的复杂度取决于采用的增量序列，不能脱离增量序列给出唯一的复杂度结论。
- 堆排序的 $O(1)$ 辅助空间，指采用迭代式筛选、在原数组内排序的实现；递归实现可能使用额外的调用栈。
- 计数排序的 $O(n+k)$ 空间，是指采用常见的稳定实现、需要计数数组和输出数组的情况。只统计频数并覆盖原数组的简化实现可以只使用 $O(k)$ 辅助空间，但不能直接稳定地排序一般的记录。

最值得优先记住的结论

- **最坏情况下仍为 $O(n\log n)$**：堆排序、二路归并排序。
- **平均 $O(n\log n)$，最坏 $O(n^2)$**：快速排序。
- **典型的 $O(n^2)$ 排序**：直接插入、折半插入、希尔排序中的某些增量方案、冒泡、简单选择。
- **可以达到 $O(n)$ 最好时间复杂度**：直接插入排序、带标志优化的冒泡排序。
- **不是基于元素比较的排序**：计数排序。

注意：折半插入排序虽然用二分查找缩短了查找插入位置的时间，但元素移动仍可能需要 $O(n^2)$，因此整体时间复杂度并没有降到 $O(n\log n)$。

二、按照基本思想分类

这部分适合用来理解算法之间的区别，也适合做选择题时快速判断。

1. 插入排序：直接插入、折半插入、希尔排序

共同思想：**维护一个已有序的部分，把新元素插入其中。**

算法	核心操作	与其他算法的区别
直接插入排序	从后向前查找插入位置，同时移动元素	查找和移动都可能耗费 $O(n)$
折半插入排序	用二分查找确定插入位置，再移动元素	减少比较次数，但不能减少大量元素移动
希尔排序	按增量分组，对组内元素进行插入排序	先局部有序，再逐渐缩小增量

记忆：直接插入和折半插入的区别是“怎么找位置”；希尔排序的特点是“先分组，再逐渐缩小增量”。

希尔排序的增量最终必须变为 1，这样最后一趟才会对整个序列进行直接插入排序。

2. 交换排序：冒泡排序、快速排序

共同思想：通过交换元素，逐步调整元素的位置。

算法	核心操作	特点
冒泡排序	比较相邻元素，逆序时交换	每一趟将较大元素逐步推向后方
快速排序	选取枢轴，通过划分将元素分到枢轴两侧	依靠划分形成子问题，再递归排序

两者的关键区别：

- **冒泡排序**：主要通过相邻元素交换实现排序。
- **快速排序**：主要通过枢轴划分实现排序。

快速排序平均性能较好，但如果每次划分都极不均衡，例如子问题规模为 0 和 $n-1$，递归深度就可能达到 $O(n)$，总时间复杂度退化为 $O(n^2)$。

3. 选择排序：简单选择、堆排序

共同思想：每一轮确定一个应该放到最终位置的元素。

算法	核心操作	特点
简单选择排序	从未排序部分中找到最小元素，放到当前首位	每一轮都要扫描剩余元素
堆排序	利用堆的性质选出最大或最小元素	可以高效地选出极值

二者都具有选择元素的思想，但实现方式不同。

简单选择排序每一轮通常需要线性扫描未排序部分，因此总时间复杂度为 $O(n^2)$。

堆排序则利用堆的结构：

1. 建立大根堆。
2. 将堆顶最大元素与当前末尾元素交换。
3. 缩小堆的有效范围。
4. 对新的堆顶进行筛选，使其恢复大根堆性质。
5. 重复上述过程。

考研重点：建堆的时间复杂度为 $O(n)$，整个堆排序的时间复杂度为 $O(n\log n)$。

4. 归并排序：二路归并

归并排序的思想是：**先分别排序，再合并有序序列。**

二路归并排序通常采用分治法：

1. 将序列分成两个子序列。
2. 分别对两个子序列进行排序。
3. 将两个有序子序列合并成一个有序序列。

它的特点是：

- 最好、平均、最坏时间复杂度均为 $O(n\log n)$。
- 标准二路归并排序需要 $O(n)$ 辅助空间。
- 标准实现是稳定排序。

5. 非比较排序：计数排序

计数排序与前面八种算法的思路明显不同。

前面的比较排序通过比较元素之间的大小关系来确定顺序；计数排序则利用关键字的取值范围，统计各个值出现的次数。

例如，待排序序列为：

$$A = [4, 2, 2, 3, 1]$$

统计结果为：

数值	出现次数
1	1
2	2
3	1
4	1

按照统计结果输出：

[1, 2, 2, 3, 4]

当关键字取值范围较小时，计数排序可以达到 $O(n + k)$ 的时间复杂度。

但是，如果关键字取值范围远大于元素数量，就可能需要很大的计数数组，因此并不适合所有数据。

三、稳定性对比：哪些稳定，哪些不稳定?

稳定性是考研中非常重要的考点。

定义： 如果两个元素的关键字相同，排序后它们的相对次序仍然不变，则称该排序算法是稳定的。

假设有三个学生：

原始次序	姓名	成绩
1	小明	90
2	小红	80
3	小华	90

按照成绩从低到高排序。

如果排序后，小明仍然排在小华前面，那么这两个成绩相同的记录保持了原来的相对次序。

稳定性速记表

算法	稳定性	主要原因
直接插入排序	稳定	不移动与待插入元素关键字相等的元素
折半插入排序	稳定	插入位置选择在相等元素之后
希尔排序	不稳定	不同分组之间可能改变相同关键字元素的相对次序
冒泡排序	稳定	相等元素不交换
快速排序	不稳定	划分过程可能改变相等元素的相对次序
简单选择排序	不稳定	交换最小元素时可能跨越相同关键字元素
堆排序	不稳定	堆顶与末尾元素交换可能改变相同关键字元素的相对次序
二路归并排序	稳定	合并时相等元素优先取自原本在前的子序列
计数排序	取决于实现	常见的累计计数加输出数组实现可以稳定排序

两个容易出错的地方

① 折半插入排序为什么稳定?

关键在于查找插入位置时，遇到相等元素，应该将新元素插入到已有相等元素的后面。

如果错误地将它插入到相等元素前面，就可能破坏稳定性。

② 简单选择排序为什么不稳定?

例如：

$$[2_a, 2_b, 1]$$

第一轮选择最小元素 1，并将它与第一个元素交换：

$$[1, 2_b, 2_a]$$

两个关键字都为 2 的元素，原本是 2_a 在前、2_b 在后，交换后次序反转，因此简单选择排序不稳定。

这个例子很适合用来理解稳定性的定义。

四、各算法的关键操作与复杂度来源

不仅要记住复杂度，还要理解它为什么是这个复杂度。

算法	关键操作	为什么是这个复杂度
直接插入排序	查找位置、移动元素	每次插入最多移动 $O(n)$ 个元素，共插入 $n-1$ 次
折半插入排序	二分查找、移动元素	查找总计 $O(n\log n)$，移动总计仍可能为 $O(n^2)$
希尔排序	分组插入排序	复杂度取决于增量序列及各趟排序的工作量
冒泡排序	相邻比较、交换	最多进行 $n-1$ 趟，每趟比较次数递减
快速排序	划分、递归排序	平均划分较均衡时为 $O(n\log n)$，极端不均衡时为 $O(n^2)$
简单选择排序	扫描未排序部分	比较次数约为 $n(n-1)/2$
堆排序	建堆、反复调整堆	建堆为 $O(n)$，后续约 n 次调整，每次至多 $O(\log n)$
二路归并排序	递归划分、合并	每层合并总工作量为 $O(n)$，共有 $O(\log n)$ 层
计数排序	统计频数、输出元素	统计和输出需要处理 n 个元素及大小为 k 的计数范围

必须理解的几个复杂度

1. 为什么堆排序建堆是 $O(n)$，而不是 $O(n\log n)$?

虽然单次向下调整最多需要 $O(\log n)$，但**并不是所有结点都需要调整这么多层**。

- 叶结点不需要向下调整。
- 靠近叶结点的结点，最多只需要调整很少几层。
- 只有少数靠近根结点的结点，才可能调整很多层。

将所有结点的调整工作量累加，建堆的总时间复杂度是：

$$\boxed{O(n)}$$

2. 为什么归并排序是 $O(n\log n)$?

归并排序每一层都需要将所有元素合并一遍，工作量约为 $O(n)$。

序列每次被分成两半，因此递归层数约为：

$$\log_2 n$$

所以总时间复杂度为：

$$\boxed{O(n\log n)}$$

3. 为什么折半插入排序仍然是 $O(n^2)$?

二分查找确实将每次查找插入位置的比较次数降到了 $O(\log n)$。

但是，插入一个元素时，为了腾出位置，后面的元素可能仍然需要整体移动。

最坏情况下，移动元素的总次数为：

$$1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2}$$

因此：

$$\boxed{T(n) = O(n^2)}$$

这也是一道很典型的复杂度辨析题。

五、原地排序与空间复杂度

原地排序通常指只使用常数级额外辅助空间完成排序；具体讨论时，需要注意递归调用栈是否计入空间复杂度。

算法	是否通常视为原地排序	辅助空间
直接插入排序	是	$O(1)$
折半插入排序	是	$O(1)$
希尔排序	是	$O(1)$
冒泡排序	是	$O(1)$
快速排序	通常是原地划分	平均递归栈 $O(\log n)$，最坏 $O(n)$
简单选择排序	是	$O(1)$
堆排序	是	迭代实现为 $O(1)$
二路归并排序	否，标准实现需要辅助数组	$O(n)$
计数排序	标准稳定实现通常需要辅助数组	$O(n+k)$

快速排序的空间复杂度怎么记?

快速排序常见的原地划分过程不需要额外创建与原数组同样大的数组，但它采用递归。

因此：

- 平均递归深度：$O(\log n)$。
- 最坏递归深度：$O(n)$。

所以不能简单地认为快速排序的所有额外空间都是 $O(1)$。

六、考研常见比较题

1. 哪些算法的最坏时间复杂度是 $O(n \log n)$?

在上述九种算法中：

- 堆排序
- 二路归并排序

快速排序的最坏时间复杂度为 $O(n^2)$，不能与它们混为一谈。

2. 哪些算法是稳定排序?

在上述常见实现中：

- 直接插入排序
- 折半插入排序
- 冒泡排序
- 二路归并排序
- 计数排序的标准稳定实现

特别注意：计数排序是否稳定取决于具体实现；希尔排序、快速排序、简单选择排序和堆排序通常不稳定。

3. 哪些算法的最好时间复杂度可以达到 $O(n)$?

- 直接插入排序：输入序列已经有序时，每个元素只需要少量比较。
- 带标志优化的冒泡排序：如果一趟扫描没有发生交换，就可以提前结束。

注意，这里的冒泡排序需要使用提前终止标志。如果没有这种优化，即使序列已经有序，也可能仍然执行多趟扫描。

4. 哪些算法需要额外的线性空间?

- 二路归并排序：通常需要 $O(n)$ 辅助数组。
- 计数排序：通常需要 $O(n + k)$ 空间，具体取决于实现。

5. 哪些算法的性能依赖输入序列或特定参数?

算法	影响因素
直接插入排序	原序列的有序程度
折半插入排序	元素移动量及序列规模
希尔排序	增量序列
冒泡排序	原序列的有序程度，以及是否使用提前终止
快速排序	枢轴选择及划分是否均衡
计数排序	关键字取值范围 k

6. 如果题目要求排序后保持相同关键字记录的相对次序，应该注意什么？

优先考虑稳定的排序算法，例如直接插入、折半插入、冒泡、标准二路归并排序，以及标准稳定计数排序。

但要注意，题目如果指定了某一种算法或具体实现，仍然需要根据实际操作判断，不能仅凭算法名称作答。

七、Mermaid：排序算法分类图

下面这张图可以直接放入支持 Mermaid 的 Obsidian 笔记中。

```
flowchart TD
    A["排序算法"] --> B["比较排序"]
    A --> C["非比较排序"]

    B --> D["插入排序"]
    B --> E["交换排序"]
    B --> F["选择排序"]
    B --> G["归并排序"]

    D --> D1["直接插入排序"]
    D --> D2["折半插入排序"]
    D --> D3["希尔排序"]

    E --> E1["冒泡排序"]
    E --> E2["快速排序"]

    F --> F1["简单选择排序"]
    F --> F2["堆排序"]

    G --> G1["二路归并排序"]
```

```
    C --> C1["计数排序"]
```

八、最后的背诵版

如果复习时间有限，建议先把下面这些内容背熟，再通过真题巩固。

算法	一句话记忆
直接插入	插入有序区，基本有序时效率高，稳定
折半插入	二分找位置，但移动仍可能是平方级，稳定
希尔排序	缩小增量、分组插入，不稳定
冒泡排序	相邻逆序就交换，优化后最好 $O(n)$，稳定
快速排序	枢轴划分，平均 $O(n\log n)$，最坏 $O(n^2)$，不稳定
简单选择	每轮选最小，比较次数固定在平方级，不稳定
堆排序	建堆 $O(n)$，排序 $O(n\log n)$，不稳定
二路归并	分治合并，始终 $O(n\log n)$，需要 $O(n)$ 辅助空间，稳定
计数排序	统计关键字频数，$O(n+k)$，适合取值范围较小的整数关键字

我建议你的复习顺序

第一优先级：

1. 九种排序算法的时间复杂度。
2. 九种排序算法的稳定性。
3. 堆排序建堆为什么是 $O(n)$。
4. 折半插入排序为什么仍然是 $O(n^2)$。
5. 快速排序为什么最坏会退化到 $O(n^2)$。
6. 归并排序的递归层数和空间复杂度。

第二优先级：

1. 每种算法的基本思想。
2. 给定序列，判断某一趟排序后的结果。
3. 根据代码判断比较次数、移动次数及算法稳定性。
4. 比较排序与非比较排序的区别。

对 408 来说，**不要只背复杂度表，还要练习根据代码和具体序列判断算法行为**。尤其是堆排序、快速排序、归并排序和各种插入排序，它们很容易以代码分析题或多项选择题的形式出现。

给我多路平衡归并与败者树的笔记。

这两个知识点在 408 数据结构的**外部排序**中非常重要，而且它们是紧密联系在一起的：多路平衡归并决定每一趟如何合并多个有序归并段，败者树则可以高效地选出下一次应该输出的最小关键字。

下面按适合放进 Obsidian 的笔记结构整理，包括基本概念、工作流程、示意图、复杂度、常见考点和易错点。

一、多路平衡归并

1. 基本概念

归并（Merge）：将多个已经有序的序列合并成一个有序序列。

多路归并（Multiway Merge）：每次将多个有序归并段合并为一个有序归并段。

多路平衡归并（Multiway Balanced Merge）：每一趟归并中，各个归并段被分配到各个归并组的方式满足平衡归并的要求，使归并过程尽可能均衡。

在外部排序中，待排序文件可能远大于内存容量，因此通常先将文件划分为若干能够装入内存的初始归并段，再利用外部存储设备完成多趟归并。

例如，假设有 4 个已经排好序的归并段：

$$R_1 = [2, 8, 15]$$

$$R_2 = [1, 7, 12]$$

$$R_3 = [3, 6, 14]$$

$$R_4 = [4, 9, 10]$$

如果采用 **4 路归并**，就可以每次从这 4 个归并段当前尚未输出的元素中，选出最小者，输出后再从对应归并段取下一个元素。

最终得到一个整体有序的序列。

2. 多路归并的核心思想

假设进行 k 路归并：

1. 每个归并段内部已经有序。
2. 从每个非空归并段中取出当前最小的未输出元素。
3. 在这 k 个候选元素中，选出最小者。
4. 将它输出到目标归并段。

5. 从被选中的归并段中补入下一个元素。
6. 重复以上过程，直到所有归并段均处理完毕。

注意：**每次只需要比较各个归并段当前的候选元素，而不需要把所有归并段重新排序。**

这也是败者树能够发挥作用的地方。

3. 为什么叫“多路”和“平衡”?

（1）多路

一次归并同时处理多个归并段。

归并方式	每次参与归并的归并段数
二路归并	2
四路归并	4
k 路归并	k

例如，8 个初始归并段：

- 二路归并：理想情况下需要 $\log_2 8 = 3$ 趟。
- 四路归并：只需要 $\log_4 8 = 1.5$ 趟，实际需要 2 趟。

更准确地说，若每趟最多将 k 个归并段合并成一个，初始有 r 个归并段，则在每趟都充分利用归并路数的理想情况下，归并趟数为：

$$\boxed{s = \lceil \log_k r \rceil}$$

这里假设允许通过适当安排归并组，使归并过程能够完成；具体外部排序实现还需要考虑归并段数量与存储设备的组织方式。

（2）平衡

在经典的多路平衡归并中，各趟归并时，归并段会尽可能均衡地分配到各个输入归并序列中。

例如，有 12 个初始归并段，进行 3 路平衡归并，可以将它们分配为：

- 输入序列 A：4 个归并段
- 输入序列 B：4 个归并段
- 输入序列 C：4 个归并段

每一趟将每个输入序列中对应的归并段进行三路归并，形成新的归并段。

需要注意，**平衡归并不意味着每个归并段的长度一定相同**，而是指归并段的分配与归并过程满足平衡组织的要求。

4. 多路归并的工作示意图

假设有 4 个初始归并段，采用 4 路归并：

```
flowchart TD
    A["归并段 R1"] --> E["4 路归并"]
    B["归并段 R2"] --> E
    C["归并段 R3"] --> E
    D["归并段 R4"] --> E

    E --> F["有序归并段 R"]
```

这里的关键操作是：每次从各个输入归并段的当前候选元素中选出最小值。

假如每次都直接扫描所有 k 个候选元素，选出最小值需要 $O(k)$ 时间。如果归并的元素总数为 N，那么选最小元素的总开销约为：

$$O(Nk)$$

当归并路数 k 较大时，这种方法效率不高。

于是就引出了败者树。

二、败者树

1. 基本概念

败者树（Loser Tree）：一种用于多路归并的选择树，它在每次比较中记录失败者，使得下一次能够高效地选出所有候选元素中的最小者。

败者树是一种特殊的树形选择结构，通常可以用完全二叉树形式的数组来实现。

它与锦标赛树的思想相似，但两者记录比赛结果的方式不同：

- **胜者树**：每个内部结点记录比赛的胜者。
- **败者树**：每个内部结点记录比赛的败者。

对于从小到大的归并，比赛中关键字较小者获胜。

败者树的目标是：**快速找到当前所有归并段的最小候选元素，并在输出一个元素后，快速更新选择结果。**

2. 为什么需要败者树?

假设现在有 8 路归并，每个归并段都有一个当前候选元素。

如果每次选最小值都进行线性扫描，需要比较约 $k-1$ 次。

当输出一个元素后，只有该元素所属归并段的候选元素发生变化，其余归并段的候选元素没有改变。

因此，没有必要每次都重新比较全部 k 个元素。

败者树利用树形结构，只需要沿着发生变化的叶结点到根结点的路径重新比较。

这样每次更新的时间复杂度就从：

$$O(k)$$

降低到：

$$\boxed{O(\log_2 k)}$$

---

3. 败者树的结构

假设进行 4 路归并，当前候选关键字分别为：

$$R_1 = 8,\quad R_2 = 3,\quad R_3 = 6,\quad R_4 = 5$$

每两个候选元素先进行比较，胜者继续向上参加比赛，败者保存在对应的内部结点中。

第一轮：

- 8 与 3 比较，3 获胜，8 失败。
- 6 与 5 比较，5 获胜，6 失败。

第二轮：

- 3 与 5 比较，3 获胜，5 失败。

最终最小值为 3，来自 R_2。

示意图如下：

```
flowchart TD
    A["8：R1"] --> E["比较 8 与 3"]
    B["3：R2"] --> E

    C["6：R3"] --> F["比较 6 与 5"]
    D["5：R4"] --> F

    E --> G["比较胜者 3 与 5"]
    F --> G

    G --> H["最终胜者：3（R2）"]
```

注意，这张图展示的是比赛过程。**败者树的内部结点记录的是各场比赛的败者，而不是胜者。**最终胜者通常单独保存，或者根据具体实现放在专门的位置。

不同教材对败者树的数组下标和最终胜者存储位置可能有不同约定，但核心思想是一致的。

---

4. 败者树如何更新?

继续使用前面的例子：

$$R_1 = 8, \quad R_2 = 3, \quad R_3 = 6, \quad R_4 = 5$$

当前最小值是 3，来自 R_2。

假设输出 3 后，R_2 读入下一个关键字 9。

候选元素变成：

$$R_1 = 8, \quad R_2 = 9, \quad R_3 = 6, \quad R_4 = 5$$

此时不需要重新比较全部 4 个元素，只需要让新元素 9 从原来 R_2 的叶结点开始，沿着通往根结点的路径重新比赛。

- 9 与原先保存在对应结点中的败者比较。
- 较小者获胜，较大者作为败者留在该结点。
- 当前胜者继续向上比赛。
- 到达根部后，得到新的全局最小值。

在这个例子中，更新后最小值是 5，来自 R_4。

考研记忆重点：每次输出最小元素后，只需要调整该元素所属叶结点到根结点的路径。

---

5. 败者树的时间与空间复杂度

设：

- k：归并路数。
- N：所有归并段中的元素总数。

（1）初始化

败者树需要建立初始比赛结构。

通常可以通过自底向上的比赛过程建立败者树，建树时间复杂度为：

$$\boxed{O(k)}$$

（2）每次选出最小元素并更新

树的高度约为：

$$\lceil \log_2 k \rceil$$

每次更新只需沿着一条根到叶的路径进行比较，因此：

$$\boxed{O(\log k)}$$

（3）整个归并过程

每个元素都需要输出一次，每次输出后至多进行一次树的调整。

因此，选择与调整的总时间复杂度为：

$$\boxed{O(N \log k)}$$

（4）空间复杂度

败者树需要保存内部结点的败者信息，通常使用与归并路数成正比的数组存储：

$$\boxed{O(k)}$$

因此，败者树的复杂度可以概括为：

操作	时间复杂度
初始化建树	$O(k)$
选出当前最小元素	$O(1)$，前提是当前胜者已保存
输出后更新败者树	$O(\log k)$
完成 N 个元素的归并	$O(N \log k)$
空间复杂度	$O(k)$

这里将“选出当前最小元素”记为 $O(1)$，是因为败者树已经维护了当前胜者；真正需要沿树进行比较的是输出元素后的更新过程。

三、多路归并与败者树的关系

两者不能混为一谈。

对比维度	多路平衡归并	败者树
本质	一种归并组织方式	一种高效的选择数据结构
主要目标	将多个有序归并段逐趟合并	快速选出当前最小候选元素
解决的问题	如何组织多路归并	如何高效完成每次选最小值
是否属于外部排序的重要内容	是	是
是否可以独立于对方使用	可以采用其他选择方法进行多路归并	也可以用于其他需要反复选出优先元素的场景
关键复杂度	理想情况下归并趟数约为 $\lceil \log_k r \rceil$	每次更新 $O(\log k)$

可以这样理解：

多路平衡归并负责安排“怎么合并”；败者树负责解决“每次合并时，谁先输出”。

如果没有败者树，也可以通过线性扫描选出最小元素，只是当归并路数较多时，效率通常会更低。

四、败者树与直接扫描的比较

假设进行 k 路归并，总共有 N 个元素需要输出。

方法	每次选最小值及更新的开销	总体选择开销
线性扫描	$O(k)$	$O(Nk)$
败者树	$O(\log k)$	$O(N \log k)$

例如，假设 k = 16。

线性扫描每次最多需要约 15 次比较；败者树每次更新只需要沿树进行约 4 层比赛。

当归并路数增大时，败者树的优势会更加明显。

不过，在真实的外部排序中，还需要考虑磁盘 I/O、缓冲区大小、归并段长度等因素。败者树主要优化的是**多路归并时的关键字选择开销**，并不会直接消除磁盘读写成本。

五、考研高频考点

1. 多路归并的归并趟数

设初始归并段数为 r，每趟最多进行 k 路归并。

在理想的平衡归并情况下，归并趟数为：

$$\boxed{s = \lceil \log_k r \rceil}$$

例如，初始有 81 个归并段，采用 3 路归并：

$$s = \lceil \log_3 81 \rceil = 4$$

因此，理想情况下需要 4 趟归并。

注意：这是归并趟数，不是败者树的高度，也不是关键字比较次数。

2. 败者树的高度

对于 k 路归并，败者树的高度为：

$$\boxed{O(\log k)}$$

每次输出元素后，只需沿着发生变化的叶结点到根结点的路径进行调整。

3. 败者树的主要优势

相比每次线性扫描全部候选元素，败者树可以将每次输出后更新选择结果的时间复杂度从：

$$O(k)$$

降低到：

$$\boxed{O(\log k)}$$

4. 多路平衡归并与败者树的区别

这是一道适合出判断题的知识点：

- 多路平衡归并是一种归并组织方式。
- 败者树是一种用于提高多路归并选最小元素效率的数据结构。
- 两者相互配合，但并不是同一个概念。

5. 外部排序的总成本

外部排序的成本不能只看比较次数，还要考虑磁盘 I/O。

如果初始归并段数量为 r，每趟采用 k 路归并，那么理想情况下归并趟数约为：

$$s = \lceil \log_k r \rceil$$

如果每一趟都需要完整读取并写回整个文件，那么归并阶段的 I/O 成本与归并趟数近似成正比。

败者树能够减少选最小值的比较开销，但**它不会直接减少既定归并组织方式下所需的归并趟数**。

六、易错点总结

易错说法	正确理解
多路归并就是败者树	错。前者是归并组织方式，后者是选择数据结构
败者树的根结点一定存放最小值	不一定。通常内部结点存败者，最终胜者单独保存；具体存储约定取决于实现
败者树每次更新都要重新比较全部元素	错。只需要调整发生变化的叶结点到根结点的路径
败者树每次更新的时间复杂度为 $O(k)$	通常为 $O(\log k)$
败者树可以减少外部排序的所有 I/O 次数	错。它主要优化选择开销，不会自动减少归并趟数
平衡归并要求所有归并段长度完全相同	错。平衡主要指归并段的分配与归并组织方式，不代表每个归并段的长度必然相同
k 路归并的趟数就是 $\log_k r$	理想情况下还要向上取整，即 $\lceil \log_k r \rceil$

七、最后的速记卡片

建议把下面这段放在笔记的最后，复习时快速回顾。

多路平衡归并

- 每次合并多个有序归并段。
- k 表示归并路数，r 表示初始归并段数。
- 理想情况下，归并趟数为 $\lceil \log_k r \rceil$。
- 外部排序需要重点考虑磁盘 I/O 成本。

败者树

- 用于多路归并时快速选出当前最小元素。
- 内部结点通常保存每场比赛的败者。
- 最终胜者单独保存或按具体实现放在专门位置。
- 建树通常为 $O(k)$。
- 每次输出后更新为 $O(\log k)$。
- 对 N 个元素完成归并，选择与调整的总开销为 $O(N \log k)$。
- 空间复杂度为 $O(k)$。

一句话记忆：

> 多路平衡归并解决“如何组织多路合并”，败者树解决“如何高效选出下一条输出记录”。